Black-Box Configuration Search for Generalized Disjunctive Programming

Albert Joon Lee [1], Woosik Kim [2], and David E. Bernal Neira [1,*]

[1]Davidson School of Chemical Engineering, Purdue University, West Lafayette, IN 47907, USA

[2]Elmore Family School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN 47907, USA

*Email: dbernaln@purdue.edu

Abstract

Selecting configurations in generalized disjunctive programming (GDP) can require repeated, costly subproblem evaluations. We evaluate RBFOpt on the external integer representation of GDP configurations. We compare it with discrete steepest descent, Branch-and-Model (BAM), and factorization-machine-based optimization with simulated annealing (FMQA-SA), alongside sampling controls. The controlled comparison separates attainment of a reference objective, configuration-evaluation cost, and optimizer overhead using stored objective responses and separately measured subproblem costs. Tests cover 21 Grid problems and four process GDP families. RBFOpt reaches the CSTR target in 45 of 60 runs, compared with 14 for discrete steepest descent, with both using a 10^6 penalty. On Grid at 100 evaluations, RBFOpt reaches the target in 81.4% of runs, compared with 70.0% for BAM and 21.4% for uniform sampling. For the dynamic process benchmark, RBFOpt reaches the target in all ten runs at the smallest tested budget of 12 evaluations, whereas discrete steepest descent first does so at the tested budget of 30. Conventional algebraic methods remain competitive when model expressions and suitable solvers are available. Black-box search completion times are estimated along recorded search paths rather than measured in online optimizer–subproblem executions. These results support RBFOpt as an alternative for GDP configuration search. Its benefit depends on the problem, failure responses, and search overhead.

Keywords: Generalized disjunctive programming; Black-box optimization; Derivative-free optimization; Surrogate modeling; Process systems engineering; Computational benchmarking.

1 Introduction

In chemical process design, evaluating a single candidate configuration can require costly optimization or simulation, making the choice of which configuration to explore next a central computational challenge. Process design involves discrete decisions, such as equipment selection, the number of processing stages, and operating modes, along with continuous variables such as flow rates, temperatures, and equipment sizes. Generalized disjunctive programming (GDP) provides a framework to represent these decisions through Boolean variables, logical propositions, and conditional constraints (Raman and Grossmann, 1994). In logic-based GDP solution methods, fixing the Boolean variables associated with a selected configuration determines which constraints remain active and yields a reduced optimization subproblem. When all discrete decisions are fixed, this subproblem can be formulated as a nonlinear programming (NLP) problem. Exploring multiple configurations may therefore require repeatedly solving NLP subproblems, which can be computationally expensive. Consequently, identifying high-quality configurations with fewer evaluations of NLP subproblems is an important objective in solving GDP problems.

Conventional GDP solution methods generally rely on mixed-integer nonlinear programming (MINLP) reformulations or logic-based decomposition. The Big-M and Hull reformulations transform disjunctive constraints into algebraic formulations (Grossmann and Lee, 2003; Raman and Grossmann, 1994), while logic-based approaches select discrete configurations and solve the corresponding reduced subproblems (Bergamini et al., 2005; Türkay and Grossmann, 1996). In logic-based approaches, the strategy used to select discrete configurations can significantly affect the number of subproblem evaluations required.

Discrete-Steepest Descent (D-SDA) was previously applied to process superstructure optimization by Liñán et al. (2020). Building on this approach, our previous work introduced the Logic-Based Discrete-Steepest Descent (LD-SDA), which represents ordered Boolean decisions using external integer variables (Ovalle et al., 2025). LD-SDA explores the resulting integer space through neighborhood search and discrete line search, evaluating selected configurations by solving their corresponding reduced optimization subproblems. However, as a local search method, LD-SDA may converge to different locally optimal solutions depending on the starting point and the selected neighborhood (Peng et al., 2024).

Surrogate-based derivative-free optimization (DFO) uses accumulated objective observations to guide new evaluations (Larson et al., 2019). RBFOpt builds radial basis function surrogates and balances predicted objective values with exploration (Costa and Nannicini, 2018). We apply it to the existing GDP external integer representation. We ask whether RBFOpt reaches reference targets more often than budget-limited D-SDA under common initial configurations and limited evaluation budgets. The comparison also includes Branch-and-Model (BAM) (Ma et al., 2023) and a variant of the factorization machine with quantum annealing (FMQA) framework (Kitai et al., 2020). We use classical simulated annealing instead of quantum annealing in this variant and denote it FMQA-SA. Uniform sampling provides a baseline without objective-dependent candidate selection.

Fewer evaluations do not necessarily imply lower computational cost, because evaluation costs vary and candidate selection incurs overhead (Moré and Wild, 2009). We therefore compare how often each method reaches the target, along with configuration-evaluation costs and search overhead. The methods share precomputed configuration responses, while response handling and stopping remain implementation-specific. The contributions are as follows:

- An evaluation of RBFOpt on the existing GDP external integer representation using 21 Grid-GDP benchmarks and four published process GDP families, with common initial configurations, evaluation budgets, and reference targets.
- An analysis of where RBFOpt improves target attainment and how configuration-evaluation costs and search overhead affect its value relative to the tested alternatives.

2 Background

2.1 Generalized Disjunctive Programming and Fixed-Configuration Evaluation

GDP represents operational and design alternatives using Boolean variables and associated algebraic constraints (Raman and Grossmann, 1994). The original continuous variables have domain $\mathcal{X} = [\underline{\mathbf{x}}, \overline{\mathbf{x}}] \subseteq \mathbb{R}^{n_x}$. The original integer variables have domain $\mathcal{Z} = [\underline{\mathbf{z}}, \overline{\mathbf{z}}] \cap \mathbb{Z}^{n_z}$. Let K index the disjunctions and D_k index the disjuncts in disjunction $k \in K$. Each disjunct $i \in D_k$ has a Boolean variable Y_{ik}. The GDP formulation is given by

$$
\begin{aligned}
\min_{\mathbf{x},\mathbf{z},\mathbf{Y}} \quad & f(\mathbf{x},\mathbf{z}) \\
\text{s.t.} \quad & \mathbf{g}(\mathbf{x},\mathbf{z}) \le \mathbf{0}, \\
& \bigvee_{i \in D_k} \begin{bmatrix} Y_{ik} \\ \mathbf{h}_{ik}(\mathbf{x},\mathbf{z}) \le \mathbf{0} \end{bmatrix}, \qquad k \in K, \\
& \Omega(\mathbf{Y}) = \text{True}, \\
& \mathbf{x} \in \mathcal{X}, \quad \mathbf{z} \in \mathcal{Z}, \quad \mathbf{Y} \in \{\text{False}, \text{True}\}^{n_y}.
\end{aligned}
\tag{GDP}
$$

Here, f is the objective function, $\mathbf{g}$ represents the global constraints, and $\mathbf{h}_{ik}$ represents the constraints associated with each disjunct. The operator $\veebar$ requires that exactly one disjunct be selected within each disjunction. Logical propositions $\Omega(\mathbf{Y})$ impose additional relationships between Boolean variables.

Following the external-variable reformulation of Ovalle et al. (2025), each ordered group of m_j Boolean alternatives is represented by an external integer variable $\xi_j \in \{1, \ldots, m_j\}$, which identifies the selected alternative. Let n_E denote the number of external integer variables, and $\mathcal{Y}_{\text{logic}}$ the set of Boolean assignments that satisfy the conditions `exactly-one`, and $\Omega(\mathbf{Y})$. The external-variable lattice and its logically admissible subset are

$$\begin{aligned} \mathcal{L}_E &= \prod_{j=1}^{n_E} \{1, \ldots, m_j\}, \\ \mathcal{E}_{\text{adm}} &= \{\boldsymbol{\xi} \in \mathcal{L}_E : \mathbf{Y}(\boldsymbol{\xi}) \in \mathcal{Y}_{\text{logic}}\}. \end{aligned} \tag{1}$$

For each logically admissible configuration $\boldsymbol{\xi}$, the corresponding Boolean variables are fixed, yielding the following reduced optimization subproblem:

$$\begin{aligned} \Phi(\boldsymbol{\xi}) = \inf_{\mathbf{x},\mathbf{z}} \quad & f(\mathbf{x}, \mathbf{z}) \\ \text{s.t.} \quad & \mathbf{g}(\mathbf{x}, \mathbf{z}) \leq \mathbf{0}, \\ & \mathbf{h}_{ik}(\mathbf{x}, \mathbf{z}) \leq \mathbf{0} \quad \text{for all } (i,k) \text{ with } Y_{ik}(\boldsymbol{\xi}) = \text{True}, \\ & \mathbf{x} \in \mathcal{X}, \quad \mathbf{z} \in \mathcal{Z}. \end{aligned} \tag{Sub}$$

Here, $\Phi(\boldsymbol{\xi})$ represents the optimal objective value of the reduced subproblem. The reduced subproblem is an NLP when all remaining decision variables are continuous. If the reduced subproblem is infeasible, we define $\Phi(\boldsymbol{\xi}) = +\infty$.

When the external-variable mapping represents all logically admissible Boolean configurations, the original GDP can be expressed as the following upper-level optimization problem (Ovalle et al., 2025):

$$\min_{\boldsymbol{\xi} \in \mathcal{E}_{\text{adm}}} \Phi(\boldsymbol{\xi}). \tag{2}$$

A logically admissible configuration may still yield an infeasible reduced subproblem.

2.2 Conventional GDP Solution Approaches

Big-M and Hull reformulations transform GDP models into mixed-integer nonlinear programming (MINLP) formulations by associating Boolean variables Y_{ik} with binary variables y_{ik} and translating logical propositions into algebraic constraints (Grossmann and Lee, 2003; Raman and Grossmann, 1994). In the Big-M reformulation, the disjunctive constraints are expressed as $\mathbf{h}_{ik}(\mathbf{x}, \mathbf{z}) \leq \mathbf{M}_{ik}(1 - y_{ik})$, where the inequality is applied componentwise. The Big-M values must be sufficiently large to render the corresponding constraints redundant when $y_{ik} = 0$, but excessively large values can weaken the continuous relaxation and cause numerical difficulties.

Hull reformulations introduce disaggregated variables, linking constraints, and bounds scaled by binary indicators (Grossmann and Lee, 2003). Disjunctive constraints are imposed on the disaggregated variables when $y_{ik} = 1$, while $y_{ik} = 0$ forces these variables to zero, making the corresponding disjunct inactive. For convex disjunctive sets, this formulation can provide a stronger continuous relaxation than Big-M, although it introduces additional variables and constraints. For nonconvex GDP problems, the Hull reformulation does not guarantee a convex reformulated model or global optimality (Chen et al., 2022).

Logic-based methods address GDP problems directly by exploiting their logical and disjunctive structure to select discrete configurations and construct the corresponding optimization subproblems. Logic-based outer approximation (LOA) alternates between a mixed-integer master problem and fixed-configuration NLP subproblems (Türkay and Grossmann, 1996). Under suitable convexity and regularity assumptions, the master problem uses linearization cuts to obtain lower bounds, while feasible solutions to the NLP subproblem provide upper bounds. For nonconvex GDP problems, these linearizations may not provide valid global lower bounds, motivating global logic-based outer approximation (GLOA), which requires globally valid relaxations and subproblem bounds. Bergamini et al. (2005) developed a GLOA approach for process networks using piecewise under- and overestimators.

GDPopt provides implementations of logic-based methods within Pyomo.GDP (Chen et al., 2022; Pyomo Development Team, n.d.). Its relaxation with integer cuts (RIC) method uses a mixed-integer linear programming (MILP) relaxation to select configurations and adds no-good cuts to exclude evaluated discrete assignments, without generating the successive outer-approximation cuts used by LOA (Pyomo Development Team, n.d.).

LD-SDA is an existing alternative that searches ordered external configurations by improving neighborhood moves and discrete line searches (Ovalle et al., 2025). It selects the best improving neighbor, continues along the resulting direction while improvement persists, and resumes neighborhood exploration. The present work uses this discrete-search structure as a reference and compares other configuration-selection strategies and their computational costs.

2.3 Derivative-Free Optimization Solvers

Derivative-free optimization (DFO) uses function observations without requiring derivatives of the objective with respect to the search variables (Larson et al., 2019). For GDP configuration search, the outer method can be derivative-free, even if its evaluation routine uses a derivative-based nonlinear solver. The upper-level search operates over external integer variables that represent the logical decisions in the GDP. Some logical restrictions are embedded in the external-variable reformulation, while additional constraints can be imposed on the configuration space. One approach to DFO uses surrogate models to guide the search. Surrogate methods seek to reduce expensive evaluations by fitting models to observed data, with computational cost arising from both objective evaluations and model-based candidate selection.

2.3.1 RBFOpt

RBFOpt is an open-source surrogate optimizer for bounded black-box problems, with support for continuous, integer, and categorical variables (Costa and Nannicini, 2018; Nannicini, 2021). Given N observed objective values in external integer configurations $\boldsymbol{\xi}^{(1)}, \ldots, \boldsymbol{\xi}^{(N)}$, RBFOpt constructs a radial basis function (RBF) surrogate model of the form

$$\widehat{\Phi}_N(\boldsymbol{\xi}) = \sum_{r=1}^{N} \lambda_r \varphi\Big(\|\boldsymbol{\xi} - \boldsymbol{\xi}^{(r)}\|_2\Big) + p(\boldsymbol{\xi}), \tag{3}$$

where $\widehat{\Phi}_N(\boldsymbol{\xi})$ approximates the objective value for a candidate configuration $\boldsymbol{\xi}$, λ_r are interpolation coefficients determined from the observed values, φ is the RBF kernel evaluated at the Euclidean distance between configurations, and $p(\boldsymbol{\xi})$ is a polynomial tail depending on the selected kernel.

Candidate selection balances low predicted values against exploration away from evaluated points. The metric stochastic response surface method (MSRSM) combines scaled surrogate predictions and distances to existing samples, changing their relative weights during the search (Regis and Shoemaker, 2007). New objective evaluations update the model.

2.3.2 Branch-and-Model (BAM)

Branch-and-Model (BAM) combines domain partitioning with local surrogate construction (Ma et al., 2023). The evaluated points define boxes, and a potentially optimal selection procedure identifies regions for further modeling and search. Within selected boxes, Automated Learning of Algebraic Models (ALAMO) constructs sparse algebraic models from nearby observations (Cozad et al., 2014). BARON optimizes these local models to propose candidates, while box-size exploration maintains broader coverage (Ma et al., 2023; Sahinidis, 1996; The Optimization Firm, 2026). Thus, BARON acts on learned surrogates within BAM; it is not solving the original GDP through this interface. Its support for integer restrictions allows for the configuration domains considered here (The Optimization Firm, 2026).

2.4 Factorization-Machine-Based Optimization

A second-order factorization machine (FM) models individual feature effects and pairwise interactions through shared latent parameters (Rendle, 2010). For a binary encoding $\mathbf{b} \in \{0,1\}^m$ of an external in-

teger configuration $\boldsymbol{\xi}$, the FM fit surrogate is

$$\widehat{\Phi}_{\mathrm{FM}}(\mathbf{b}) = \beta_0 + \sum_{i=1}^{m} \beta_i b_i + \sum_{i=1}^{m-1} \sum_{j=i+1}^{m} \langle \mathbf{a}_i, \mathbf{a}_j \rangle b_i b_j, \tag{4}$$

where $\widehat{\Phi}_{\mathrm{FM}}(\mathbf{b})$ denotes the predicted training response, m is the number of binary features, β_0 is the intercept, β_i is the linear coefficient of b_i, and $\mathbf{a}_i \in \mathbb{R}^r$ is its latent vector of dimension r. The inner product $\langle \mathbf{a}_i, \mathbf{a}_j \rangle$ determines the pairwise interaction coefficient. The model parameters are learned from the evaluated configurations and their corresponding training responses.

For binary inputs, $b_i^2 = b_i$, the fitted model is a binary quadratic model. Excluding the constant term β_0 gives the following quadratic unconstrained binary optimization (QUBO) problem:

$$\min_{\mathbf{b} \in \{0,1\}^m} \mathbf{b}^{\mathsf{T}} \mathbf{Q} \mathbf{b}, \qquad Q_{ij} = \begin{cases} \beta_i, & i = j, \\ \langle \mathbf{a}_i, \mathbf{a}_j \rangle, & i < j, \\ 0, & i > j. \end{cases} \tag{5}$$

The QUBO describes the learned surrogate, not an exact algebraic reformulation of the GDP. Moreover, a binary string need not decode to a logically admissible configuration in the domain.

The factorization machine with quantum annealing (FMQA) framework combines FM surrogate fitting with annealing-based candidate generation (Kitai et al., 2020). Based on observed evaluations, the method fits the FM, samples binary candidates from the resulting quadratic model, and evaluates a selected decoded configuration through the objective-evaluation routine before updating the training data.

The FMQA simulated annealing variant (FMQA-SA) uses classical simulated annealing to sample binary candidates from the learned QUBO. Temperature-dependent acceptance of uphill moves supports exploration, while gradual cooling favors lower-energy solutions (Kirkpatrick et al., 1983). In this study, no quantum hardware was used.

3 Experimental Methods

3.1 Benchmark problems: Grid-GDP

We consider 21 two-dimensional test functions adapted from the Quantum Hamiltonian Descent benchmark collection (Wu Group, n.d.). Each is sampled at 31 equally spaced values per coordinate, giving 961 configurations. For each benchmark function, let $[\ell_x, u_x]$ and $[\ell_y, u_y]$ denote the coordinate bounds, and let $I = J = \{0, \ldots, 30\}$ be the corresponding index sets. The Grid optimizers receive $(i, j) \in \{0, \ldots, 30\}^2$ directly. Relative to the one-based convention in Section 2.1, $(\xi_1, \xi_2) = (i + 1, j + 1)$ is only a relabeling of alternatives. `FMQA-SA` encodes each Grid index in five unsigned bits and rejects decoded values of 31. The physical coordinates are defined as

$$a_i = \ell_x + \frac{i(u_x - \ell_x)}{30}, \qquad b_j = \ell_y + \frac{j(u_y - \ell_y)}{30}. \tag{6}$$

Let f denote the original analytical objective function and let L and U be its minimum and maximum over the 31×31 lattice. Both constants are obtained by exhaustive preprocessing of all 961 lattice points prior to any search. A normalized target tolerance of 10^{-5} corresponds to $10^{-5}(U - L)$ in raw objective units, so equal normalized tolerances do not need to define equally large target sets. Supplementary Table S4 (Section S6) gives L, U, and target-set sizes for all functions. Each benchmark function is represented by

the following GDP:

$$\begin{aligned}
\min_{x,y,Y^x,Y^y} \quad & F(x,y) = \frac{f(x,y) - L}{U - L} \\
\text{s.t.} \quad & \bigvee_{i \in I} \begin{bmatrix} Y_i^x \\ x = a_i \end{bmatrix}, \qquad \bigvee_{j \in J} \begin{bmatrix} Y_j^y \\ y = b_j \end{bmatrix}, \\
& \text{ExactlyOne}(Y_0^x, \ldots, Y_{30}^x), \\
& \text{ExactlyOne}(Y_0^y, \ldots, Y_{30}^y), \\
& \ell_x \le x \le u_x, \qquad \ell_y \le y \le u_y, \\
& Y_i^x, Y_j^y \in \{\text{False}, \text{True}\}.
\end{aligned} \tag{7}$$

The two exclusive disjunctions contain 62 coordinate-selection Boolean indicators. The objective retains the explicit mathematical expression. Consequently, the algebraic solvers receive the expression and its disjunctive structure, whereas the DFO methods observe only objective values returned at their selected coordinates.

Fixing both coordinate disjunctions fixes x and y, leaving no remaining original continuous degrees of freedom. Thus, the measured fixed-point `IPOPT` (Wächter and Biegler, 2006) evaluations characterize fresh model preparation, solver invocation, and verification, rather than a difficult continuous optimization subproblem. The unfixed objective functions remain nonlinear and generally nonconvex, so Big-M and Hull produce MINLP representations. The complete mathematical definitions and coordinate domains of the 21 benchmark functions are provided in Supplementary Sections S1–S2 (Table S1) (SECQUOIA, 2026b; Wu Group, n.d.).

3.2 Process GDP benchmark problems

The process benchmarks include a CSTR network (Liñán et al., 2020), a 17-stage distillation column (Ghouse et al., 2018), and a small batch plant (Kocis and Grossmann, 1988). We use their GDP formulations reported by Ovalle et al. (2025), and the nine-stage nonlinear mode-switching model of Peng et al. (2024). Their objectives minimize the total reactor volume (`CSTR`), condenser and reboiler duties, plus the active tray cost (`Column`), the investment cost (`Small Batch`), and the negative integral of the squared state (`Dynamic`). The external variables represent the reactor/feed cutoff and recycle location (`CSTR`), reflux and boilup positions (`Column`), parallel unit counts (`Small Batch`), and final stages of modes 1 and 2 (`Dynamic`). For these process benchmarks, fixing the external choices leaves only continuous variables, so the resulting subproblems are NLPs. The `CSTR` superstructure contains N potential reactor positions, where $N \in \{5, 10, 15, 20, 25, 30\}$. We denote each `CSTR` instance by its size parameter N, such as `CSTR30` for $N = 30$. `Dynamic` uses 30 finite elements per stage and three Radau collocation points. The reference configurations in external variable coordinates are (N, N) for `CSTR`, $(13, 4)$ for `Column` (physical trays $(14, 5)$), $(2, 2, 1)$ for `Small Batch`, and $(1, 4)$ for `Dynamic`. The full GDP formulations of the `CSTR`, `Column`, and `Small Batch` benchmarks are provided in Appendices A.1–A.3 of Ovalle et al. (2025), respectively, while the `Dynamic` formulation is described in Section IV.C of Peng et al. (2024). External-variable mappings for the three design cases are provided in Appendices B.1, B.2, and B.4 of Ovalle et al. (2025). The `CSTR` volume is reported in liters and the `Small Batch` investment in the source model's dollars. The `Column` uses the source model's weighted cost proxy based on duties and active trays. The model sizes and unit conventions are given in the Supplementary Section S5 (Table S3). `Dynamic` is an abstract benchmark with no calibrated physical units for state or time. Minimizing its negative squared-state integral favors greater integrated state magnitude and tests mode sequencing coupled to dynamic equations.

Table 1 summarizes the original GDP and DFO candidate domains, the number of configurations with `IPOPT`-accepted objective values, the number of configurations with measured evaluation times, and the evaluation budgets. The process searches use frozen objective tables from fixed-configuration `IPOPT` evaluations; only accepted finite objectives are used as valid lookup objectives. The objective, acceptance status, and point cost are taken from the same recorded evaluation attempt for the same model, CSTR size, and external-variable tuple. Measured coverage includes all configurations in each DFO candidate box, including unsuccessful and structural-rejection evaluations. The evaluation budget B limits the total number of objective evaluations, including initialization, repeated calls, and unsuccessful evaluations. The full GDP box

precedes the logical and algebraic feasibility restrictions. `Column` searches $\{7,\ldots,15\}\times\{1,\ldots,9\}$ (81 coordinates) within its original box $\{1,\ldots,15\}^2$ (225 coordinates). Other cases use their full boxes. We selected the `Column` search box to limit the scope of the experiment, without a specific engineering or optimization criterion. All DFO methods use this box, and the `Column` conclusions apply only to it. Configurations outside it are not asserted to be infeasible. For `CSTR`, recycle/feed ordering admits $N(N+1)/2$ logical candidates, but not all yield objective values satisfying the solver acceptance criteria defined in Section 3.4. The `CSTR` comparisons do not isolate scalability: evaluation budgets change with size. `Dynamic` permits $z_{E,1} < z_{E,2}$ or $(z_{E,1}, z_{E,2}) = (9, 9)$. `Small Batch` uses all 27 configurations in $\{1,2,3\}^3$, whose decisions count parallel mixer, reactor, and centrifuge units. Of these configurations, 15 are rejected during feasibility-based bound tightening (FBBT) preprocessing.

Table 1: Process GDP domains, `IPOPT`-accepted lookup coverage, measured point-cost coverage, evaluation budgets, and reference objective values. Model sizes and unit conventions are given in Supplementary Table S3.

(a) External-variable domains

Model	Full GDP box	DFO candidate box	Full GDP box size	DFO box size
`Column`	$\{1,\ldots,15\}^2$	$\{7,\ldots,15\}\times\{1,\ldots,9\}$	225	81
`CSTR` N	$\{1,\ldots,N\}^2$	$\{1,\ldots,N\}^2$	N^2	N^2
`Small Batch`	$\{1,2,3\}^3$	$\{1,2,3\}^3$	27	27
`Dynamic`	$\{1,\ldots,9\}\times\{2,\ldots,9\}$	$\{1,\ldots,9\}\times\{2,\ldots,9\}$	72	72

(b) Lookup coverage and evaluation budgets

Model	Finite lookup	Measured coordinates	Evaluation budget (B)	f_{ref}
`Column`	29	81	16	19346.1248
`CSTR5`	15	25	8	3.0620146
`CSTR10`	54	100	15	2.8895370
`CSTR15`	120	225	22	2.8293941
`CSTR20`	210	400	29	2.7994064
`CSTR25`	325	625	36	2.7814826
`CSTR30`	464	900	43	2.7695697
`Small Batch`	12	27	10	167427.6571
`Dynamic`	37	72	12,15,20, 30,37,50	-212.0088
Total	1,266	2,455	–	–

The 2,455 measured keys cover the complete DFO boxes. The 700 primary process runs (`D-SDA`, `RBFOpt`, `BAM`, `FMQA-SA`, and uniform sampling; 14 model–budget conditions and ten seeds per condition) visit 2,040 distinct model/size/coordinate keys. This union counts each key once across methods, budgets, and seeds.

3.3 Search protocol and compared algorithms

3.3.1 Shared initialization and evaluation budgets

All methods use precomputed configuration responses during the search. For process cases, accepted responses are objective values from `IPOPT` under the initialization and acceptance rules in Section 3.4. They need not equal the theoretical subproblem value $\Phi(\boldsymbol{\xi})$.

Each Grid-GDP benchmark uses ten initial designs and evaluation budgets $B \in \{10, 30, 50, 70, 100\}$. The primary Grid comparison comprises four search implementations and two sampling controls, giving $21 \times 10 \times 5 \times 6 = 6{,}300$ runs. The process benchmarks comprise 14 model–budget conditions, each evaluated with ten paired seeds, giving 140 runs per method and 700 runs across the four search implementations and uniform sampling (Table 1). For each problem-budget-seed combination, three distinct initial configurations are generated using seeded Latin hypercube sampling, mapped to integer coordinates, and shared in the same order across all methods and sampling controls. This common design controls the first three evaluations; subsequent sampling, response handling, and stopping rules follow the individual implementations. All

process cases use seeds 0–9, with no replacement of failed starts. `Column` seed 0 has three unaccepted initial configurations; the resulting `D-SDA` execution failure remains in the ten-run denominator.

Methods may terminate before exhausting the evaluation budget B. Table 2 separates the algorithm stopping from the worker watchdog; detailed recorded options, seeds, versions, and cache behavior are provided in Supplementary Section S4 and its machine-readable companion files. All objective callbacks consume the evaluation budget, including initialization, repeated calls, and unaccepted evaluations. Internal cache accesses without objective callbacks are not counted as evaluations. Evaluation acceptance and target attainment are assessed using the recorded evaluation status and original objective values, rather than method-specific penalties. Unaccepted evaluations return 10^6 to `D-SDA` and `RBFOpt` in all process cases. `BAM` receives `NaN` in all process cases. `FMQA-SA` retains infinite raw values and separately reports 10^6. These penalties do not replace the recorded solver status or establish mathematical infeasibility.

Table 2: Configuration summary for the compared DFO implementations

Method	Stopping and target information	Candidate generation / seed
`D-SDA`	Budget or no improving move. Grid also stops at the known target and attempts ranked initial starts; process uses one best accepted start.	Fixed L_∞ order, first tie, improvement $> 10^{-12}$; common design seed.
`RBFOpt`	Budget and native iteration/stall limits; no benchmark target supplied. Native clock limit 10^{30} s (effectively disabled).	MSRSM; genetic candidate search; other `Bonmin`/`IPOPT` surrogate solves retained; job seed.
`BAM`	Native remaining budget $B - 3$ and package stopping defaults; no benchmark target supplied.	`ALAMO`/`BARON` local models; common evaluated starts, plus configured sampling.
`FMQA-SA`	Budget or 1,800 s method limit; no benchmark target supplied.	Rank-eight FM and classical annealing; separate seeded model, annealing, and fallback streams.
Uniform	Budget; no benchmark target supplied.	Uniform unseen integer configurations after the shared starts.
`BAM`-prefix control	Budget; Grid only.	Preserve `BAM` callbacks 4–13, then sample uniformly without replacement.

Original process jobs and the additional process comparisons have 2,100 s worker watchdogs; the additional Grid comparisons have the watchdog disabled. Grid `BAM` trace watchdogs were also disabled, while Grid `FMQA-SA` used 2,100 s. Surrogate solver calls contribute to search time but are distinct from `IPOPT` evaluations of the original process subproblems. Exact recorded dictionaries and remaining environment limitations are documented in Supplementary Sections S4–S4.1.

3.3.2 Budget-limited D-SDA

We use a budget-limited implementation of the previously developed `D-SDA` (Liñán et al., 2020), applying its neighborhood search and discrete line search over the GDP external integer lattice as in LD-SDA (Ovalle et al., 2025). It evaluates the unit L_∞ neighborhood in fixed enumeration order, selects the best improving neighbor, and continues along the resulting direction through a discrete line search. An improvement requires an objective decrease greater than 10^{-12}, with ties resolved by the first enumerated neighbor. The exploration of the neighborhood resumes from the last improvement point. If the evaluation budget B is exhausted, the best solution found is retained.

For the process benchmarks, `D-SDA` starts with the best feasible configuration among the three initial points evaluated. For Grid-GDP benchmarks, initial configurations are ranked by objective value and successive descents are attempted until the evaluation budget is exhausted or the stored target is reached within an absolute tolerance of 10^{-12}. This algorithmic stopping tolerance is different from the 10^{-5} assessment tolerance in Equation (8), which is applied to all methods. For all process benchmarks, `D-SDA` searches within the declared integer bounds without precomputed feasibility filtering, using the callback accounting

and unaccepted-response policy stated above.

3.3.3 RBFOpt

`RBFOpt` uses MSRSM, automatic RBF selection, and the default genetic candidate-search option of version 4.3.0 (Costa and Nannicini, 2018; Nannicini, 2021). The settings are `rbf=auto` and `global_search_method=genetic`, with the recorded defaults `ga_base_population_size=400` and `ga_num_generations=20`. The implementation determines the effective population size from the search dimension (Supplementary Section S4). All external variables are integers, and the three shared initial configurations are supplied with the additional package initialization disabled. These shared-start and budget settings define the experimental protocol; the implementation is not described as an entirely default package invocation. Other surrogate-minimization operations use standalone `Bonmin` 1.8.9 (Bonami et al., 2008) and `IPOPT` 3.14.19 (Wächter and Biegler, 2006) executables. These calls contribute to search time, but not the objective-evaluation budget. Unaccepted evaluations follow the common policy stated above.

3.3.4 BAM

`BAM` uses local algebraic surrogate models optimized by `BARON` to generate candidate configurations (Ma et al., 2023; The Optimization Firm, 2026). All external variables are specified as integers using `integrality=1`. The three shared initial configurations are evaluated and supplied by `x_evaluated` and `f_evaluated`, without an additional starting point (`x0 = None`). The remaining evaluation budget is set to `max_evals`$=B-3$, while other algorithmic settings retain their defaults. As specified in the shared evaluation protocol, unaccepted evaluations of process benchmarks return `NaN` to `BAM`. After the three shared evaluations, `BAM` uses ten predetermined Grid points from a Sobol sequence, a sampling sequence designed to distribute points evenly across the search domain. The first two of these points account for all additional target hits in a total budget of ten evaluations. Objective evaluations, including repeated calls, are counted independently by the external ledger. For the process experiments, callback B returns its value normally, and `BAM` terminates through its native evaluation limit. All 140 `BAM` process runs use this stopping procedure with the paired seeds and frozen `IPOPT` tables.

3.3.5 FMQA-SA

`FMQA-SA` reuses the PyTorch-based FM training and binary quadratic model (BQM) construction code from the `QHD_Data` branch of `SECQUOIA/bbo_via_fmqa` (SECQUOIA, n.d.a). At each adaptive iteration, a second-order rank-eight FM with identity activation is initialized with linear and latent weights drawn from a normal distribution with mean zero and standard deviation 0.01, and zero bias. The model is trained on all previously evaluated configurations for 1,000 full-batch epochs using Adam (Kingma and Ba, 2015), mean squared error, and a learning rate of 0.01. Finite precomputed responses are min–max scaled to $[0, 1]$ (or zero when all finite values are identical), while non-finite evaluations receive a training target of 2.0. For reporting and incumbent tracking, non-finite raw objective values are represented by 10^6.

Each integer coordinate ξ_j is encoded in unsigned binary using $\lfloor \log_2 u_j \rfloor + 1$ bits, where $u_j > 0$ is its upper limit. The fitted FM is converted to a QUBO, with its coefficients scaled by the maximum absolute entry of the quadratic matrix. Binary candidates are generated locally on CPU using D-Wave's classical `SimulatedAnnealingSampler` from `dwave-samplers` 1.6.0, with 100 reads, 1,000 sweeps, and a geometric inverse-temperature schedule (D-Wave Systems, n.d.). The lowest-energy decoded candidate within the coordinate bounds that has not been previously evaluated is selected; if none is available, a candidate is drawn uniformly from the remaining unevaluated configurations. The logical constraints of GDP and the feasibility of NLP are not screened prior to objective evaluation, so infeasible configurations may still consume the evaluation budget. The new observation is appended, and FM fitting and candidate generation repeat until the evaluation budget B or the 1,800 s method time limit is reached.

3.3.6 Sampling controls

Uniform sampling evaluates the same three initial configurations and then draws uniformly without replacement from the remaining declared integer domain. It uses a separate deterministic random stream,

no objective-dependent selection, and no logical or NLP-feasibility screening before evaluation. The Grid BAM-prefix control repeats BAM's ten predetermined sampling points in the same order after the three shared starts, then samples uniformly without replacement, stopping at the budget. We checked the preserved prefix across all 1,050 BAM Grid runs; it includes the corner and center evaluations responsible for the early target hits. This control tests the observed fixed sampling design and does not reconstruct proprietary BAM internals. Every callback consumes the same budget and saved point cost under the common acceptance and target rules.

3.4 Execution environment and algebraic comparisons

The benchmark performance experiments were conducted on the Dantzig server, equipped with an AMD EPYC 7643 processor (48 cores, 2.3 GHz) and 1 TB RAM. All optimization methods and numerical libraries were configured for single-threaded execution. The software environment included Pyomo 6.10.0 (Bynum et al., 2021), RBFOpt 4.3.0, BAM 5.0.0, PyTorch 2.8.0 (Paszke et al., 2019), and HiGHS 1.12.0 (Huangfu and Hall, 2018). The additional comparisons used the same recorded host, Python/package versions, and Bonmin/IPOPT binaries as their historical references. The Dantzig auxiliary-solver banners identify Bonmin 1.8.9 with Cbc 2.10.12 and IPOPT 3.14.19, and a standalone IPOPT 3.14.19 executable; both are Linux x86_64 builds with ASL 20231111. We used the General Algebraic Modeling System (GAMS) 52.2.0 (GAMS Development Corporation, 2025) to call IPOPT for configuration evaluations and BARON for algebraic comparisons. BAM uses engine 2026.9.10, build LNX-int-64 dated 2026-09-10 17:59:06. The algebraic comparisons use BARON 25.11.17; the individual ALAMO and BARON versions inside BAM are not reported by its runtime information and remain unidentified.

DFO benchmarking. Configuration-evaluation costs were measured separately from searches and combined with recorded method times as described in Section 3.6.

Fixed-disjunction NLP evaluations. We fixed the original Boolean GDP decisions and evaluated the resulting subproblems using IPOPT 3.14.19 with MUMPS 5.8.1 (Amestoy et al., 2001). For the Grid-GDP benchmarks, all 20,181 configurations were evaluated individually, while the process benchmarks used fixed-configuration evaluations of the corresponding process models. IPOPT used a tolerance of 10^{-8} and a solver time limit of 60 s per configuration. Process acceptance requires a finite objective, maximum absolute constraint, and variable-bound violation of at most 10^{-5}, and termination as optimal, locallyOptimal, or globallyOptimal. The solver status must not be error or aborted; the interface must report SOLVESTAT=1 and MODELSTAT$\in \{1, 2\}$. Time-limited candidates remain unaccepted even when feasible. Supplementary Section S4.1 specifies the residual checks and initialization artifacts. Unaccepted responses do not establish mathematical infeasibility. CSTR sizes 5–30 use the mass-balance/zero-recycle initialization. Column and Small Batch use their saved initialization files, and Dynamic uses fresh clones of the unsolved discretized template.

Algebraic GDP comparisons. Conventional comparisons include GDPOpt RIC, GLOA, and LOA, as well as Big-M and Hull reformulations. GDPOpt uses HiGHS for MILP master problems and BARON for nonlinear subproblems (Chen et al., 2022; Pyomo Development Team, n.d.). The Big-M and Hull reformulations are solved using BARON as the MINLP solver. All algebraic methods use the original GDP domains, a time limit of 1,800 s per instance, and a relative optimality gap of 10^{-5}.

3.5 Target attainment and cumulative success profiles

Let f_{best} denote the lowest feasible objective value observed so far within the evaluation budget, and let f_{ref} denote the reference objective value. For process benchmarks, f_{best} is computed from evaluations satisfying the solver acceptance criteria in Section 3.4. A run reaches the reference target when

$$|f_{\text{best}} - f_{\text{ref}}| \leq 10^{-5} \max\{1, |f_{\text{ref}}|\}. \tag{8}$$

Target attainment measures agreement with the reference value rather than certifying global optimality. For the process experiments, f_{ref} is the lowest accepted objective value obtained from the precomputed fixed-configuration NLP evaluations. Runs that terminate with an execution failure are counted as unsuccessful, even if the target was observed earlier. For process runs, the reported solution is the accepted configuration with the lowest objective recorded by the wrapper within budget. Its coordinates and objective are preserved in the final `runs.csv` summary, including when first observed at callback B. This solution need not equal the native optimizer's returned incumbent. Terminal accepted observations are retained under the same rule in the `RBFOpt` and uniform comparisons. For the full-process-GDP comparison, a feasible objective at or below $f_{\text{ref}} + 10^{-5}\max\{1, |f_{\text{ref}}|\}$ counts as achievement, so solutions better than the reference are also included.

For each evaluation budget B, let $\mathcal{R}_B$ denote the set of problem-seed instances compared across methods, and let s identify a solution method. For each instance $r \in \mathcal{R}_B$, H_{sBr} equals one if the run reaches the target and zero otherwise. Process indicators use the wrapper-reported best-accepted configuration defined above, with execution failures assigned zero. Let E_{sBr} be the index of the first objective evaluation at which the target is reached, with $E_{sBr} = +\infty$ if the target is not reached. At a threshold of N objective evaluations, where $0 \le N \le B$, the absolute evaluation-count profile is

$$C_{sB}^{\text{eval}}(N) = \frac{1}{|\mathcal{R}_B|} \sum_{r \in \mathcal{R}_B} \mathbf{1}\{H_{sBr} = 1,\ E_{sBr} \le N\}. \tag{9}$$

Here, $\mathbf{1}\{\cdot\}$ equals one when the stated conditions hold and zero otherwise, and $|\mathcal{R}_B|$ is the number of problem-seed instances. Thus, $C_{sB}^{\text{eval}}(N)$ gives the fraction of runs that reach the target within N evaluations. For the Grid-GDP benchmarks, $|\mathcal{R}_B| = 210$ at each budget; for each process benchmark and budget, $|\mathcal{R}_B| = 10$. All runs, including target misses, remain in the denominator.

Let $\widehat{T}_{sBr}$ denote the estimated full-run completion time, combining recorded search time with separately measured configuration-evaluation costs. Lookup-time adjustments and model-preparation costs are accounted for as described in Section 3.6. For a time threshold $t \ge 0$, measured in seconds, the absolute completion-time profile is

$$C_{sB}^{\text{time}}(t) = \frac{1}{|\mathcal{R}_B|} \sum_{r \in \mathcal{R}_B} \mathbf{1}\{H_{sBr} = 1,\ \widehat{T}_{sBr} \le t\}. \tag{10}$$

This profile gives the fraction of runs that reach the target and have an estimated full-run completion time no greater than t. A successful run is counted at its full completion estimate, not at the time of its first target attainment.

Experiments at different evaluation budgets are analyzed separately; a larger-budget trajectory need not extend a smaller-budget trajectory. Both profiles use absolute evaluation counts or times rather than solver-relative cost ratios (Dolan and Moré, 2002). The evaluation count profile is related to the data profiles (Moré and Wild, 2009), but uses raw rather than dimension-normalized evaluation counts. Algebraic results are reported separately because their iterations and subsolver calls are not DFO objective evaluations, and only one run is reported for each problem-method pair.

3.6 Runtime accounting and uncertainty

For a recorded lookup-based run with n objective evaluations and measured lookup-call times, the estimated full-run completion time is

$$\widehat{T}_{\text{complete}} = T_{\text{recorded}} - \sum_{q=1}^{n} t_q^{\text{lookup}} + \sum_{q=1}^{n} c(\boldsymbol{\xi}^{(q)}) + t_{\text{prep}}. \tag{11}$$

Here, T_{recorded} is the measured runtime of the search based on lookup, t_q^{lookup} is the measured lookup-processing interval of its qth objective call, and $\boldsymbol{\xi}^{(q)}$ is the external integer configuration evaluated in that call. For the process experiments, this interval covers coordinate processing and in-memory lookup, but excludes callback-file writing; the CSV is loaded before the search clock starts. The point cost $c(\boldsymbol{\xi}^{(q)})$ is measured separately for the same benchmark model, problem size and configuration, including configuration-specific preparation, preprocessing, invocation of the solver when required and validation. The term t_{prep} accounts

for the preparation of a single model that is not already included in the point costs. Retained search time includes surrogate fitting, candidate generation, auxiliary optimization, and recording performed within the method clock. For historical process runners, T_{recorded} is the `interval dfo _wall _s` ending after the optimization process termination and status checks, immediately before `result.json` is written. For the additional `RBFOpt` and sampling runs, it is the recorded search-stage interval, with later incumbent extraction and export recorded separately and excluded. Callback logging within that interval is included. The later `summarize()` pass reconstructs the best accepted configuration from the callback records and writes `runs.csv`; these final result-extraction and summary-writing operations are excluded from the completion estimates. Throughout this manuscript, full-run completion denotes completion of this recorded search stage with reconstructed evaluation costs, excluding final result extraction and export. Every recorded objective call contributes its point cost, including initialization, repeated visits, and preprocessing rejections; the same stored measurement is used each time a configuration is revisited. The reported completion-cost distributions are conditional on this frozen point-cost inventory. They reflect variation in search paths and recorded search overhead, but do not estimate repeated-measurement uncertainty in configuration-evaluation costs.

Grid-GDP timers. The Grid-GDP enumeration and objective normalization are excluded from the timing analysis, and $t_{\text{prep}} = 0$ because configuration-specific preparation is included in the point costs. For `D-SDA` on the Grid-GDP benchmarks, lookup-call times were not recorded, so the method runtime is retained without subtracting lookup time.

Process timers. Each recorded point cost is preserved with its objective and status. Measured rejection costs cover the structural exclusions: 435 for `CSTR30`, 17 for `Column`, and 35 for `Dynamic`. These coordinates follow the same model preparation and logical-rejection paths, with no NLP solver invocation. Their positive total processing times are measured instead of set to zero. Costs are joined by the complete model/size/coordinate key, never by the coordinate tuple alone. No cross-size `CSTR` timing substitution or successful repetition selection is used. For `Dynamic`, the original model-template construction and discretization cost is included once per run through t_{prep}; additional campaign-level preparation is not charged repeatedly. All in-budget evaluations have measured point costs, and estimated full-run completion times are available for all 700 primary process runs. The sole execution failure, a `Column D-SDA` initialization failure, retains its recorded cost and contributes zero target success. These estimates retain the recorded lookup trajectories rather than measuring online optimizer–NLP executions.

Algebraic GDP completion times are measured separately and are not augmented with the DFO point costs. Grid-GDP algebraic timings include per-run preparation, reformulation, solution, and validation, but exclude shared-template preparation and lattice enumeration. Full-process-GDP timings include model-template preparation.

Uncertainty. Supplementary Section S7 defines pointwise 95% exact binomial intervals for the ten-seed success fractions. These intervals assume independent seed-level trials under the fixed protocol and are not simultaneous intervals. Paired comparisons resample intact functions for Grid and paired seeds within each process condition (Supplementary Section S10).

4 Computational Results and Discussion

4.1 Search Performance on Grid-GDP Benchmarks

`RBFOpt` attains more Grid targets than budget-limited `D-SDA` at every tested evaluation budget (Table 3). At $B = 100$, it reaches 171 of 210 targets (81.4%), compared with 104 (49.5%) for `D-SDA` and 45 (21.4%) for uniform sampling. From $B = 30$ onward, it has the highest aggregate attainment among the compared methods.

At $B = 10$, `BAM` and its prefix control each reach 111 targets, compared with 42 for `RBFOpt`. All 111 `BAM` hits occur before surrogate-guided selection: 11 during shared initialization and 100 at two predetermined Sobol points. At $B = 100$, `BAM` adds 23 hits over the prefix control, indicating a benefit from its subsequent search.

Target-set sizes vary across functions (Supplementary Table S4). Excluding Xin-She Yang 3 and the nearly redundant sampled Bohachevsky 2 case preserves the $B = 100$ ordering (Supplementary Section S10).

Table 3: Grid-GDP reference-target hit percentages from the recorded trajectories. Each cell contains 210 runs (21 functions and ten seeds). The evaluation budget includes the three shared initial evaluations, and repeated objective callbacks are counted toward the evaluation budget. `BAM`'s 111 hits at $B = 10$ all precede surrogate-guided selection (11 shared-initialization hits and 100 at two predetermined Sobol points). Per-function integer hit counts at $B = 100$ are in Supplementary Table S6.

Budget B	`D-SDA`	RBFOpt	BAM	`FMQA-SA`	Uniform	BAM-prefix control
10	6.2	20.0	52.9	7.6	6.7	52.9
30	35.7	61.9	54.8	22.4	11.9	54.8
50	42.4	73.3	57.6	36.2	13.8	56.7
70	48.1	76.7	62.9	48.6	17.1	57.1
100	49.5	81.4	70.0	61.4	21.4	59.0

Figure 1 shows the cumulative fraction of runs attaining the reference target as a function of the number of objective evaluations, revealing when successful searches first reach the target. Each panel is a separate budget experiment, so trajectories at a larger evaluation budget are not assumed to extend those at a smaller evaluation budget. `FMQA-SA` reaches 129 of 210 targets at $B = 100$, compared with 45 for uniform sampling.

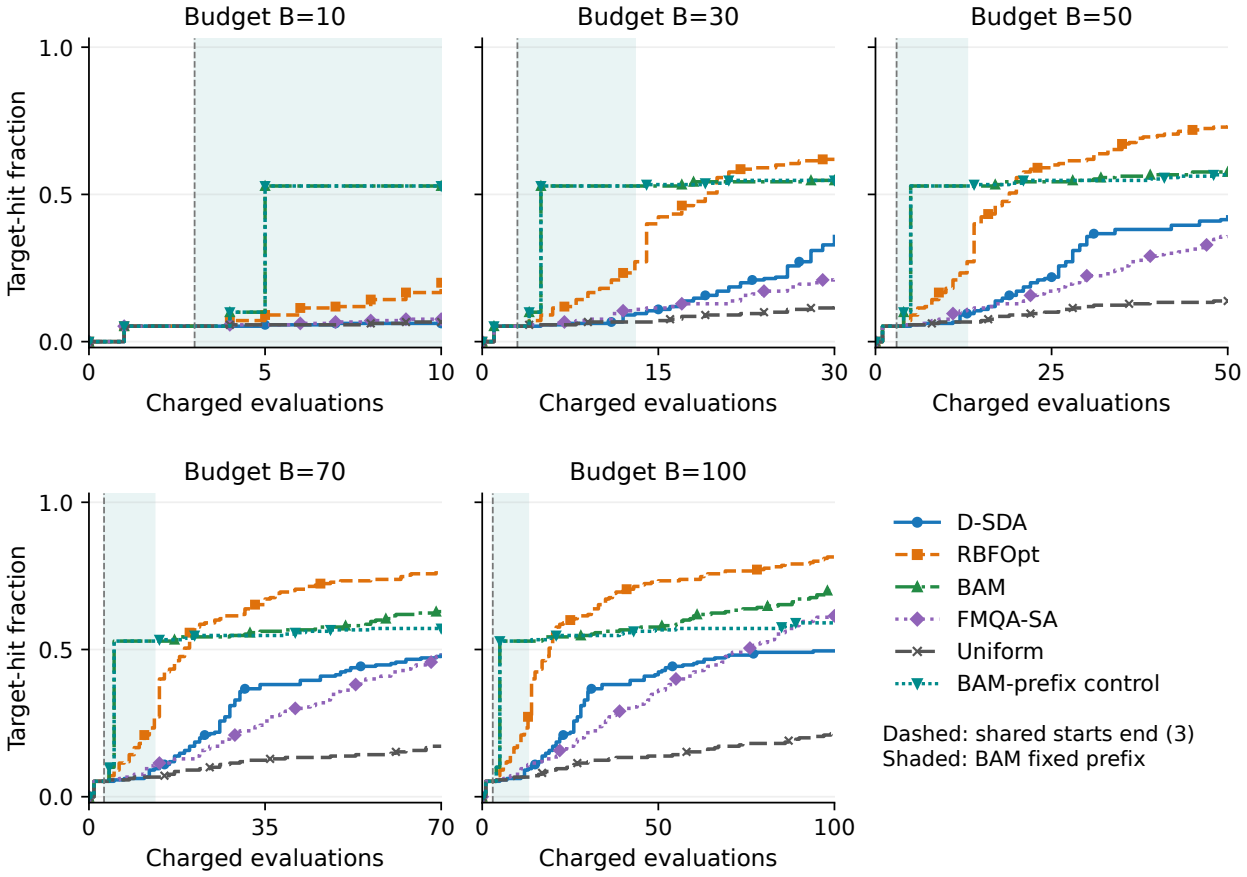


Figure 1: Absolute evaluation-count cumulative success profiles for four search implementations and two sampling controls. The horizontal axis is the number of objective callbacks to first target attainment; the vertical axis is the fraction of all 210 problem–seed instances reaching the target within that count. Initial and repeated callbacks count; unsuccessful runs remain in the denominator, and counts are not normalized by another solver. The panels use evaluation budgets of 10, 30, 50, 70, and 100; colors, line styles, and markers distinguish methods. The gray dashed line at three evaluations marks the end of the shared initialization. Shading from callback 4 through the smaller of 13 and the budget marks the preserved `BAM` sampling prefix.

4.2 Computational Comparison on Grid-GDP Problems

Higher target attainment comes with additional search cost (Table 4). At $B = 100$, `RBFOpt` has a median estimated full-run completion time of 70.81 s, compared with 24.14 s for `D-SDA` and 64.42 s for `BAM`. `D-SDA` has negligible search overhead and can stop before exhausting the budget, while `RBFOpt` spends a median of 7.579 s on search. For `FMQA-SA`, the median search component is 62.04 s and the median completion estimate is 124.13 s. Because Grid configurations fix both original coordinates, their measured evaluation costs primarily reflect model preparation, solver invocation, and verification.

Table 4: Grid-GDP DFO results at $B = 100$, with 210 runs per method. All time columns are medians over target hits and misses. Component medians need not sum to the completion median. The times cover the full run rather than only the evaluations up to the first hit.

Method	Target hits	Median full-run completion (s)	Median summed point cost (s)	Median retained search time (s)
`D-SDA`	104/210	24.14	24.14	0.0032
RBFOpt	171/210	70.81	60.84	7.5790
BAM	147/210	64.42	63.97	0.4301
`FMQA-SA`	129/210	124.13	62.34	62.0359
Uniform	45/210	60.51	60.51	0.0026
BAM-prefix control	124/210	61.81	61.80	0.0026

`D-SDA` and `BAM` nevertheless have zero observed hits on four and two functions, respectively; `RBFOpt` and `FMQA-SA` have at least one hit on every function at that evaluation budget. Uniform has seven zero-hit functions, and the prefix control has two.

The algebraic comparisons use one run per problem and method, and their iterations and subsolver calls are not equivalent to DFO callbacks. The algebraic solver families and configurations are described in Section 3.4. Each `GDPOpt` method (RIC, GLOA, and LOA) attains the reference target on 18 of 21 functions; all three miss Hölder table, Michalewicz, and Shubert (historical variant). but why? This is the difference between a high school report and a paper; here you are just reporting things as if, without any motivation Target attainment can coexist with termination at a time limit, as in all three `GDPOpt` runs for Three-hump camel and the Lévy 13/RIC run. Consequently, the measured completion time of a target-attaining run must be distinguished from a completed global-optimality certification.

Big-M/`BARON` attains four of 21 targets and Hull/`BARON` five of 21. Each has 16 unsupported functions because the configured `BARON` interface does not accept the required sine/cosine expressions. These entries are solver-compatibility limitations, not optimization timeouts or mathematical failures of either reformulation. Among the five supported functions, target-attaining runs have measured worker completion times between approximately 0.55 and 1.30 s; Hull attains all five targets, whereas Big-M misses Styblinski–Tang.

These supported cases demonstrate that access to the algebraic structure can be highly competitive, while full-suite hit fractions additionally reflect implementation coverage.

4.3 Process GDP Case Studies

4.3.1 Target attainment

Across the six `CSTR` sizes, `RBFOpt` reaches 45 of 60 targets, compared with 14 for `D-SDA` (Table 5). It has more target hits at every tested size under the stated size-dependent budgets. For `Dynamic`, `RBFOpt` reaches all ten targets at every tested budget, starting at $B = 12$, where `D-SDA` reaches four. `D-SDA` reaches all ten from $B = 30$ onward.

The advantage is problem-dependent. On `Column`, both methods reach five of ten targets within the restricted 81-configuration box. On `Small Batch` at $B = 10$, `D-SDA` reaches eight targets, compared with five for `RBFOpt`.

`FMQA-SA` reaches 42 of 140 process targets, compared with 36 for uniform sampling. Its aggregate gain over uniform is smaller here than on Grid. Supplementary Section S4.2 reports fallback counts, and Section S10 reports the FMQA diagnostics.

Table 5: Reference-target attainment in the IPOPT-table process experiments. Entries are target hits of the wrapper-reported best-accepted configuration divided by all ten paired seeds at evaluation budget B. The main criticism is, which tables HAVE to appear in the main body, and which are just supplementary. In general, the paper is too long and says too little for how many pages it has

Case	B	D-SDA	RBFOpt	BAM	FMQA-SA	Uniform
Column	16	$5/10^{F1}$	5/10	0/10	4/10	0/10
CSTR5	8	2/10	8/10	10/10	3/10	4/10
CSTR10	15	2/10	6/10	1/10	2/10	1/10
CSTR15	22	2/10	7/10	1/10	1/10	2/10
CSTR20	29	3/10	7/10	2/10	1/10	2/10
CSTR25	36	3/10	8/10	2/10	1/10	1/10
CSTR30	43	2/10	9/10	2/10	1/10	1/10
Small Batch	10	8/10	5/10	2/10	6/10	4/10
Dynamic	12	4/10	10/10	0/10	2/10	0/10
Dynamic	15	5/10	10/10	3/10	3/10	3/10
Dynamic	20	8/10	10/10	3/10	3/10	3/10
Dynamic	30	10/10	10/10	4/10	4/10	4/10
Dynamic	37	10/10	10/10	7/10	5/10	5/10
Dynamic	50	10/10	10/10	10/10	6/10	6/10

Superscript Fn denotes n execution failures, which remain in the denominator and do not count as hits. The only failure is a Column D-SDA initialization failure. All 70 primary groups use seeds 0–9. All methods use the stated initialization and stopping policies. An accepted target configuration at callback B counts because its coordinates and objective are preserved in the final wrapper summary. BAM also returns that value to its native optimizer; the other methods retain the wrapper stop (Supplementary Section S4.3). The reference is the accepted IPOPT-table minimum, not a global-optimality certificate.

Figure 2 shows the first target-attaining evaluation on successful recorded runs, with misses and execution failures retained in the denominator.

4.3.2 Computational cost

For Dynamic at $B = 50$, search overhead contributes 42.67% of the mean estimated completion time for RBFOpt, compared with less than 0.2% for D-SDA. Dynamic D-SDA executes 221 callbacks across ten $B = 50$ runs, compared with 500 for each of RBFOpt, BAM, and FMQA-SA. Its lower cost reflects shorter paths as well as low search overhead; unsuccessful candidates are evaluated and charged within the common box.

Figure 3 presents target attainment against estimated full-run completion time for the nine illustrative conditions in Figure 2. For Dynamic at $B = 30$, D-SDA and RBFOpt both reach ten targets, with median completion estimates of 26.61 and 34.19 s, respectively. RBFOpt already reaches all ten at $B = 12$ with a median of 12.29 s. Thus the comparison depends on whether the evaluation budget is fixed or selected separately for each method.

Supplementary Table S5 reports median estimated full-run completion times for all 14 conditions. For CSTR30, RBFOpt reaches nine targets with a median completion estimate of 101.96 s. D-SDA reaches two targets at 53.08 s, and BAM reaches two at 115.41 s.

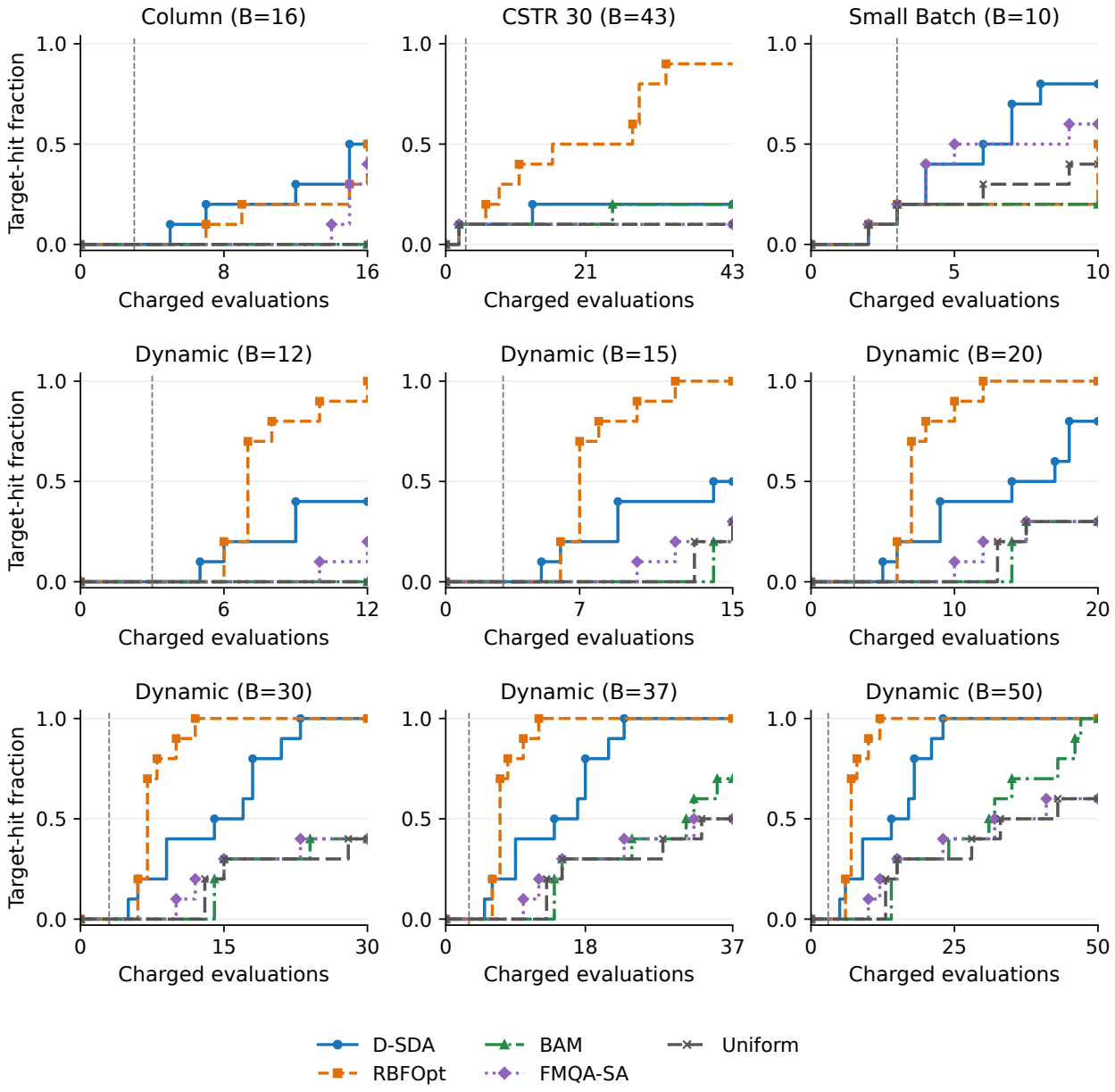


Figure 2: Cumulative target attainment versus charged evaluations for nine illustrative process conditions, using the wrapper-reported success definition in Section 3.5. Every method–condition group retains all ten seeds, including execution failures. Initialization, unsuccessful evaluations, and repeated callbacks consume the budget. Steps mark the first accepted target observation on successful runs. The gray vertical line at three evaluations marks the end of the shared initialization. Line styles and markers distinguish methods. All 14 conditions appear in Table 5.

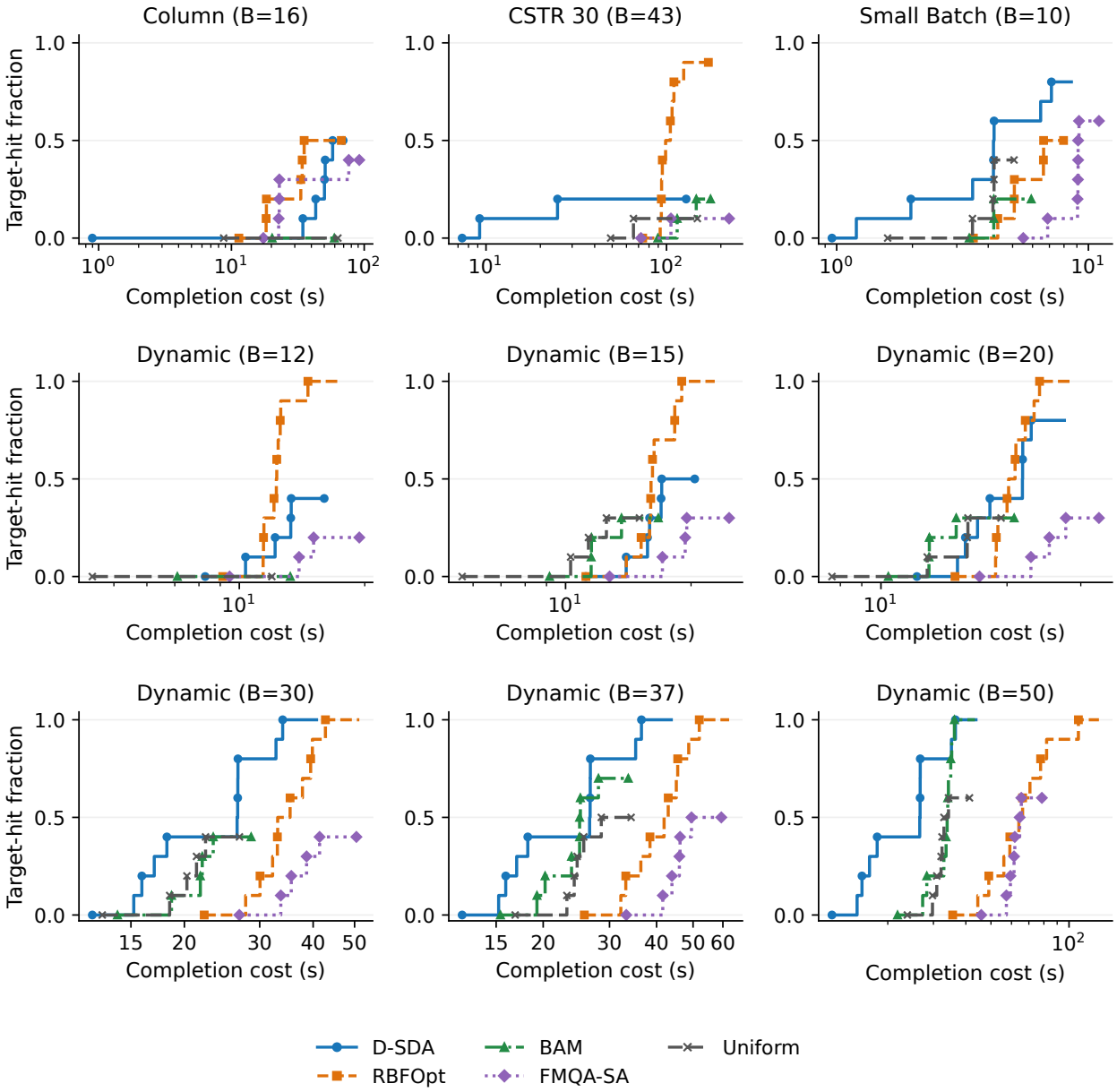


Figure 3: Absolute cumulative target-attainment profiles for nine process conditions. Reconstructed search-completion costs exclude the final wrapper result-extraction and summary-writing pass; see Section 3.6. The horizontal axis is reconstructed full-run completion time on a logarithmic scale; all ten seeds remain in each denominator. Only runs whose wrapper-reported best-accepted configuration attains the target contribute steps. Target misses and execution failures contribute no success. Every run has a complete cost estimate. These are fixed-path estimates, not first-hit-time curves or solver-relative ratios. All 14 conditions are summarized in Supplementary Table S5.

Table 6 compares five algebraic methods on nine process GDP models. All 45 saved solutions are numerically feasible under the original-GDP residual test with threshold 10^{-5}. The target-attainment counts are 3/9 for RIC, 1/9 for GLOA, 4/9 for LOA, 7/9 for Big-M, and 6/9 for Hull. These single-run results do not establish success probabilities or global optimality.

All five methods attain the `Small Batch` reference. Big-M and Hull complete in 0.55 and 0.57 s, respectively, compared with median DFO completion estimates ranging from 3.49 to 9.09 s. This demonstrates the competitiveness of algebraic methods when the formulation is available, although the different information access and timing protocols prevent direct speedup claims.

These single-run algebraic comparisons distinguish numerical feasibility, reference-target attainment, solver termination, and optimality certification. All six `CSTR` GLOA runs report `optimal` with numerically feasible solutions, but miss the `IPOPT`-table references, which are not independently certified global optima. Reevaluation of the archived GLOA and RIC vectors in the same original GDP, using its objective and variable-name mapping, gives a better RIC objective for every size. The gaps are 2.23–27.57% of the RIC values, approximately 2,228–27,571 times the comparison tolerance $10^{-5}\max\{1, |f_{\mathrm{RIC}}|\}$. Together with `Column` Big-$M$ and `CSTR30` Hull, eight raw `optimal` reports are target misses, with their original objectives, times, and termination labels retained. The six GLOA lower bounds equal their returned objectives and exceed the checked RIC values; Table 6 applies the same bound-consistency flag to all methods. The smaller Hull exceedances for `CSTR5`, `CSTR10`, and `CSTR15` are only about 1.00, 1.81, and 1.54 times the comparison tolerance, respectively. These are numerical-consistency observations, not demonstrated software defects or evidence of general solver inferiority; the cause in `GDPOpt` or `BARON` remains unresolved. LOA's time-limited `CSTR15` run also returns an objective below the lookup reference, illustrating why target attainment and certification are assessed separately.

A separate `CSTR15` check on Dantzig using `IPOPT` 3.14.19 retained witness W03's discrete configuration and starting values (Supplementary Section S9) (SECQUOIA, 2026c). The returned objective was 2.8293940632857595, with maximum unscaled original-GDP violation 1.11×10^{-16}, passing a 10^{-8} residual target. The recorded GLOA lower bound still exceeds this objective by 0.6931579445, approximately 24,498 times the audit tolerance, so the numerical discrepancy persists at the tighter threshold without establishing exact feasibility or its cause. In contrast, the Hull lower bound exceeds the tightened objective by only 5.15×10^{-4} times the comparison tolerance, below the flagging threshold. This additional check leaves the historical performance values and audit classifications unchanged.

Table 6: Full-process-GDP results: objective (measured completion seconds, including model-template preparation). `GDPOpt` uses `BARON` for nonlinear subproblems and `HiGHS` for master problems; Big-M and Hull use `BARON`. Feasibility means numerical feasibility under the original-GDP residual test at 10^{-5}, not exact feasibility.

Model	RIC	GLOA	LOA	Big-M	Hull
`CSTR5`	3.0619840 (31.27)	$3.1301981^{\dagger\ddagger}$ (3.76)	3.0619840 (31.82)	3.0620136 (9.12)	$3.0620146^{\dagger}$ (14.86)
`CSTR10`	2.8895081 (214.06)	$3.4479640^{\dagger\ddagger}$ (4.56)	2.8895081 (234.13)	2.8894835 (123.74)	$2.8895358^{\dagger}$ (126.47)
`CSTR15`	2.9660174^{TL} (1798.83)	$3.5225520^{\dagger\ddagger}$ (26.19)	2.8293506^{TL} (1799.38)	2.8293940^{TL} (1799.00)	$2.8293941^{\dagger}$ (940.36)
`CSTR20`	2.9095042^{TL} (1799.04)	$3.6161230^{\dagger\ddagger}$ (6.79)	2.8895081^{TL} (1797.72)	2.7994063^{TL} (1798.72)	2.7994064^{TL} (1797.44)
`CSTR25`	2.8594370^{TL} (1797.05)	$3.6478181^{\dagger\ddagger}$ (85.20)	2.7951055^{TL} (1797.46)	2.7814826^{TL} (1797.67)	2.7814826^{TL} (1797.41)
`CSTR30`	2.8379523^{TL} (1792.52)	$3.1543081^{\dagger\ddagger}$ (718.80)	2.7844377^{TL} (1798.92)	2.7695698^{TL} (1797.31)	$2.8093862^{\dagger\ddagger}$ (1466.60)
`Column`	22271.549^{TL} (1797.54)	22271.549^{TL} (1798.15)	22271.549^{TL} (1797.77)	$19853.723^{\dagger\ddagger}$ (34.36)	20837.080^{TL} (1797.17)
`Dynamic`	-119.383^{TL} (1797.71)	-119.383^{TL} (1797.55)	-119.383^{TL} (1797.84)	-207.626^{TL} (1797.51)	-28.597^{TL} (1797.77)
`Small Batch`	167427.657 (12.86)	167427.657 (14.49)	167427.657 (5.11)	167427.657 (0.55)	167427.657 (0.57)

Raw termination is `optimal` unless TL denotes `maxTimeLimit` with a saved feasible incumbent. Reference-target classification uses the lookup-reference values in Table 1. ‡: reference target not attained despite the reported `optimal`; a better independently checked numerically feasible solution exists under the stated residual test, so the report is not treated as an independent optimality certificate. This is a target miss, not an execution failure. †: reported lower bound exceeds the best checked saved numerically feasible value f by more than $10^{-5}\max\{1, |f|\}$; the unrounded bound-consistency results and comparison solutions are provided in Supplementary Section S9, Tables S7–S8, and `algebraic_audit/table8_bound_audit.csv` in the companion data. Near-tolerance cases are discussed in the text. The same rule applies to all methods. All 45 saved vectors pass the original-GDP feasibility recheck at 10^{-5}, which does not establish optimality. Each method/model has one run; no success probability is inferred. Times are full-run completion times.

4.4 Integrated Numerical Discussion

The results support RBFOpt for configuration search under limited evaluation budgets, particularly on CSTR and at low budgets on Dynamic. Its stronger Grid attainment is consistent with this result, while Column and Small Batch show that the advantage is problem-dependent.

Target attainment must be weighed against search overhead and stopping behavior. For Dynamic at $B = 30$, D-SDA uses a median of 23.5 callbacks, compared with 30 for RBFOpt. Their median post-target counts are 6.5 and 23, respectively. Thus full-run completion costs include work after the target is first observed.

These findings concern the tested configuration domains and fixed local-solver responses. The simultaneous changes in CSTR size and budget do not isolate scalability. The restricted Column box and the acceptance policy further limit extrapolation.

Algebraic formulations provide competitive reference results when supported, particularly for Small Batch. Their expression coverage and access to model structure differ from those of DFO, and the unresolved optimality reports discussed in Section 4.3 cannot serve as independent certificates.

5 Conclusions

This study evaluated RBFOpt for GDP configuration search using external integer variables. It reached 45 of 60 CSTR targets under the tested budgets, compared with 14 for D-SDA. On Dynamic, it reached all ten targets at 12 evaluations, while D-SDA first did so at the tested budget of 30. Across the Grid-GDP and process GDP benchmarks, higher target coverage did not consistently yield lower estimated full-run completion times. Performance depends on the problem, failure responses, and search overhead.

These results support RBFOpt as an alternative for GDP configuration search under limited evaluation budgets. D-SDA performed better on Small Batch. Algebraic approaches remain useful alternatives when model expressions and suitable solvers are available.

The findings are conditional on finite candidate domains and frozen local-solver responses. Completion costs are retrospective estimates, not measured online optimizer–NLP times. Ten-seed hit rates do not establish 99% solution-time guarantees, and the conflicting algebraic optimality reports remain unresolved. Transfer to broader configuration spaces and live NLP evaluation remains to be tested.

6 Acknowledgement

The authors acknowledge and express sincere gratitude to the Davidson School of Chemical Engineering at Purdue University for providing a supportive research environment and funding.

Data and code availability

The code, frozen evaluation data, archived execution records, and analysis scripts are maintained in SECQUOIA/Black-Box-for-GDP; the repository remains private at this revision and public release is planned after submission, subject to the repository administrator enabling access (SECQUOIA, 2026a,b). Supplementary Section S8 identifies the archived inputs, including the CSTR finite-penalty runs and updated CSTR30 evaluations.

References

Amestoy, P.R., Duff, I.S., Koster, J., L'Excellent, J.Y., 2001. A fully asynchronous multifrontal solver using distributed dynamic scheduling. SIAM Journal on Matrix Analysis and Applications 23, 15–41. doi:10.1137/S0895479899358194.

Bergamini, M.L., Aguirre, P., Grossmann, I., 2005. Logic-based outer approximation for globally optimal synthesis of process networks. Comput. Chem. Eng. 29, 1914–1933. doi:10.1016/j.compchemeng.2005.04.003.

Bonami, P., Biegler, L.T., Conn, A.R., Cornuéjols, G., Grossmann, I.E., Laird, C.D., Lee, J., Lodi, A., Margot, F., Sawaya, N., Wächter, A., 2008. An algorithmic framework for convex mixed integer nonlinear programs. Discrete Optimization 5, 186–204. doi:10.1016/j.disopt.2006.10.011.

Bynum, M.L., Hackebeil, G.A., Hart, W.E., Laird, C.D., Nicholson, B.L., Siirola, J.D., Watson, J.P., Woodruff, D.L., 2021. Pyomo: Optimization Modeling in Python. volume 67 of *Springer Optimization and Its Applications*. 3 ed., Springer. doi:10.1007/978-3-030-68928-5.

Chen, Q., Johnson, E.S., Bernal, D.E., Valentin, R., Kale, S., Bates, J., Siirola, J.D., Grossmann, I.E., 2022. Pyomo.GDP: an ecosystem for logic based modeling and optimization development. Optim. Eng. 23, 607–642. doi:10.1007/s11081-021-09601-7.

Clopper, C.J., Pearson, E.S., 1934. The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika 26, 404–413. doi:10.1093/biomet/26.4.404.

Costa, A., Nannicini, G., 2018. RBFOpt: an open-source library for black-box optimization with costly function evaluations. Math. Program. Comput. 10, 597–629. doi:10.1007/s12532-018-0144-7.

Cozad, A., Sahinidis, N.V., Miller, D.C., 2014. Learning surrogate models for simulation-based optimization. AIChE J. 60, 2211–2227. doi:10.1002/aic.14418.

D-Wave Systems, n.d. dwave-samplers: Classical algorithms for solving binary quadratic models. URL: https://github.com/dwavesystems/dwave-samplers/releases/tag/1.6.0. Version 1.6.0; classical CPU simulated annealing; accessed October 15, 2026.

Dolan, E.D., Moré, J.J., 2002. Benchmarking optimization software with performance profiles. Math. Program. 91, 201–213. doi:10.1007/s101070100263.

GAMS Development Corporation, 2025. General Algebraic Modeling System (GAMS). URL: https://www.gams.com/. release 52.2.0, Fairfax, VA, USA.

Ghouse, J.H., Chen, Q., Zamarripa, M.A., Lee, A., Burgard, A.P., Grossmann, I.E., Miller, D.C., 2018. A comparative study between gdp and nlp formulations for conceptual design of distillation columns, in: Computer Aided Chemical Engineering, pp. 865–870. doi:10.1016/B978-0-444-64241-7.50139-7.

Grossmann, I.E., Lee, S., 2003. Generalized convex disjunctive programming: Nonlinear convex hull relaxation. Comput. Optim. Appl. 26, 83–100. doi:10.1023/A:1025154322278.

Huangfu, Q., Hall, J.A.J., 2018. Parallelizing the dual revised simplex method. Mathematical Programming Computation 10, 119–142. doi:10.1007/s12532-017-0130-5.

Kingma, D.P., Ba, J., 2015. Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR). URL: https://arxiv.org/abs/1412.6980.

Kirkpatrick, S., Gelatt, Jr., C.D., Vecchi, M.P., 1983. Optimization by simulated annealing. Science 220, 671–680. doi:10.1126/science.220.4598.671.

Kitai, K., Guo, J., Ju, S., Tanaka, S., Tsuda, K., Shiomi, J., Tamura, R., 2020. Designing metamaterials with quantum annealing and factorization machines. Phys. Rev. Res. 2, 013319. doi:10.1103/PhysRevResearch.2.013319.

Kocis, G.R., Grossmann, I.E., 1988. Global optimization of nonconvex mixed-integer nonlinear programming (minlp) problems in process synthesis. Industrial & Engineering Chemistry Research 27, 1407–1421. doi:10.1021/ie00080a013.

Larson, J., Menickelly, M., Wild, S.M., 2019. Derivative-free optimization methods. Acta Numerica 28, 287–404. doi:10.1017/S0962492919000060.

Liñán, D.A., Bernal, D.E., Ricardez-Sandoval, L.A., Gómez, J.M., 2020. Optimal design of superstructures for placing units and streams with multiple and ordered available locations. part i: A new mathematical framework. Computers & Chemical Engineering 137, 106794. doi:10.1016/j.compchemeng.2020.106794.

Ma, K., Rios, L.M., Bhosekar, A., Sahinidis, N.V., Rajagopalan, S., 2023. Branch-and-Model: a derivative-free global optimization algorithm. Comput. Optim. Appl. 85, 337–367. doi:10.1007/s10589-023-00466-3.

Moré, J.J., Wild, S.M., 2009. Benchmarking derivative-free optimization algorithms. SIAM J. Optim. 20, 172–191. doi:10.1137/080724083.

Nannicini, G., 2021. On the implementation of a global optimization method for mixed-variable problems. Open J. Math. Optim. 2, 1–25. doi:10.5802/ojmo.3.

Ovalle, D., Liñán, D.A., Lee, A., Gómez, J.M., Ricardez-Sandoval, L., Grossmann, I.E., Bernal Neira, D.E., 2025. Logic-Based Discrete-Steepest Descent: A solution method for process synthesis Generalized Disjunctive Programs. Comput. Chem. Eng. 195, 108993. doi:10.1016/j.compchemeng.2024.108993.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. PyTorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems, pp. 8024–8035.

Peng, Z., Lee, A., Bernal Neira, D.E., 2024. Addressing discrete dynamic optimization via a logic-based discrete-steepest descent algorithm, in: 2024 IEEE 63rd Conference on Decision and Control (CDC), IEEE. pp. 1664–1669. doi:10.1109/CDC56724.2024.10886477.

Pyomo Development Team, n.d. GDPopt logic-based solver. URL: https://pyomo.readthedocs.io/en/6.10.0/explanation/solvers/gdpopt.html. Version 6.10.0; accessed October 20, 2026.

Raman, R., Grossmann, I.E., 1994. Modelling and computational techniques for logic based integer programming. Comput. Chem. Eng. 18, 563–578. doi:10.1016/0098-1354(93)E0010-7.

Regis, R.G., Shoemaker, C.A., 2007. A stochastic radial basis function method for the global optimization of expensive functions. INFORMS J. Comput. 19, 497–509. doi:10.1287/ijoc.1060.0182.

Rendle, S., 2010. Factorization machines, in: 2010 IEEE International Conference on Data Mining, pp. 995–1000. doi:10.1109/ICDM.2010.127.

Sahinidis, N.V., 1996. BARON: A general purpose global optimization software package. J. Glob. Optim. 8, 201–205. doi:10.1007/BF00138693.

SECQUOIA, 2026a. Black-Box-for-GDP: additional genetic-search and sampling comparisons. URL: https://github.com/SECQUOIA/Black-Box-for-GDP/tree/6e083eb49b497ce4fea90131e18cc0459db682e8/results/review_comparison_20261020. Analysis snapshot on review/additional-comparisons, accessed October 20, 2026; original execution provider 38d917ef75ef6767c2a9f238d88be19bf0b89160; private pending administrator-enabled public release.

SECQUOIA, 2026b. Black-Box-for-GDP: analytical Grid-GDP and process benchmark archive. URL: https://github.com/SECQUOIA/Black-Box-for-GDP/tree/cb70771b2037301bb6d0e38a2362d49b53fcda7d. Snapshot of main; commit cb70771b2037301bb6d0e38a2362d49b53fcda7d, accessed October 19, 2026.

SECQUOIA, 2026c. Black-Box-for-GDP: Dantzig GAMS/IPOPT CSTR15 tightened-feasibility validation. URL: https://github.com/SECQUOIA/Black-Box-for-GDP/tree/5c124ffdfb2fc0b54673ae784d368f4b2afce973/results/review_cstr15_dantzig_20261021. Result and analysis snapshot, accessed October 21, 2026; single server call at execution revision 70bf17ca95e5ee1b98069e494c22fcafd6398960; historical benchmark records preserved.

SECQUOIA, 2026d. Black-Box-for-GDP: limited FMQA and numerical-feasibility diagnostics. URL: https://github.com/SECQUOIA/Black-Box-for-GDP/tree/15663134704b7bb1d4804c94c70edabaffbf8ecd/results/review_diagnostics_20261020. Diagnostic snapshot accessed October 20, 2026; reconstructed fits are distinct from unsaved historical weights; earlier snapshot preceding the separate CSTR15 tightened-feasibility follow-up.

SECQUOIA, 2026e. Black-Box-for-GDP: primary comparison figures, tables, and Process coverage audit. URL: https://github.com/SECQUOIA/Black-Box-for-GDP/tree/3c60f4707b454a9d3e4330466ec5f9372b418f54. Publication analysis snapshot, accessed October 21, 2026; includes the 700-run, 2,050-key coverage audit; original execution revisions preserved.

SECQUOIA, n.d.a. bbo_via_fmqa: Black-box optimization via factorization machines and quantum annealing. URL: https://github.com/SECQUOIA/bbo_via_fmqa/tree/eb3a20d4b5a3773d17ff31d72a35d51f58e84e56. QHD_Data branch; commit eb3a20d4b5a3773d17ff31d72a35d51f58e84e56; upstream PyTorch FM and BQM code; accessed October 19, 2026.

SECQUOIA, n.d.b. Black-Box-for-GDP: historical QHDOpt GDP formulation. URL: https://github.com/SECQUOIA/Black-Box-for-GDP/blob/11ba8728a107d6d16ed63e72d334a66662f9ebc1/qhdopt_gdp.py. Historical source; commit 11ba8728a107d6d16ed63e72d334a66662f9ebc1; accessed October 19, 2026.

The Optimization Firm, 2026. BAM python interface documentation: bam-solver 5.0.0. URL: https://pypi.org/project/bam-solver/5.0.0/. Version 5.0.0, released September 11, 2026; executed engine 2026.9.10; accessed October 20, 2026.

Türkay, M., Grossmann, I.E., 1996. Logic-based MINLP algorithms for the optimal synthesis of process networks. Comput. Chem. Eng. 20, 959–978. doi:10.1016/0098-1354(95)00219-7.

Wächter, A., Biegler, L.T., 2006. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. Mathematical Programming 106, 25–57. doi:10.1007/s10107-004-0559-y.

Wu Group, n.d. Quantum Hamiltonian descent: 2D test functions. https://jiaqileng.github.io/quantum-hamiltonian-descent/nonconvex2d.html. Source collection for the 21 adapted finite grids used here; accessed September 29, 2026.

Supplementary Material: Benchmark Definitions, Execution Protocol, and Uncertainty

This supplement to *Black-Box Configuration Search for Generalized Disjunctive Programming* provides benchmark definitions, execution and acceptance rules, model sizes, seed-level uncertainty, and the numerical evidence underlying the algebraic bound-consistency checks. The Grid-GDP functions are adapted from the Quantum Hamiltonian Descent collection; each instance is defined by its implemented expression and domain rather than its conventional name (SECQUOIA, 2026b; Wu Group, n.d.).

S1 Coordinate domains and normalization

For each row of Table S1, both endpoints are included and the physical coordinates are

$$a_i = \ell_x + \frac{i}{30}(u_x - \ell_x), \qquad b_j = \ell_y + \frac{j}{30}(u_y - \ell_y), \qquad i, j \in \{0, \dots, 30\}. \tag{S1}$$

The raw objective f is defined below. The minimized objective is

$$F(x, y) = \frac{f(x, y) - L}{U - L}, \qquad L = \min_{i,j} f(a_i, b_j), \quad U = \max_{i,j} f(a_i, b_j). \tag{S2}$$

Thus L and U are obtained by exhaustive preprocessing of the 961 lattice points, not from the continuous optimum. All trigonometric arguments are in radians; $e = \exp(1)$ and π is the usual circular constant.

Table S1: Grid-GDP benchmark domains. All intervals include both endpoints and have equally spaced coordinates. Each row defines one independent problem with 961 configurations and 62 Boolean indicators. Historical function expressions and domains are preserved (SECQUOIA, 2026b; Wu Group, n.d.).

Function	x domain	y domain	Grid
Ackley	$[-5, 5]$	$[-5, 5]$	31×31
Alpine 1	$[0, 10]$	$[0, 10]$	31×31
Alpine 2	$[0, 1000]$	$[0, 1000]$	31×31
Bohachevsky 2	$[-100, 100]$	$[-100, 100]$	31×31
Three-hump camel	$[-5, 5]$	$[-5, 5]$	31×31
Csendes (squared variant)	$[-1, 1]$	$[-1, 1]$	31×31
Deflected corrugated spring	$[0, 10]$	$[0, 10]$	31×31
Drop-Wave	$[-5.12, 5.12]$	$[-5.12, 5.12]$	31×31
Easom	$[-10, 10]$	$[-10, 10]$	31×31
Griewank	$[-600, 600]$	$[-600, 600]$	31×31
Hölder table	$[-10, 10]$	$[-10, 10]$	31×31
Hosaki	$[0, 5]$	$[0, 6]$	31×31
Lévy	$[-10, 10]$	$[-10, 10]$	31×31
Lévy 13	$[-10, 10]$	$[-10, 10]$	31×31
Michalewicz	$[0, \pi]$	$[0, \pi]$	31×31
Rastrigin	$[-5.12, 5.12]$	$[-5.12, 5.12]$	31×31
Rosenbrock	$[-2, 2]$	$[-1, 3]$	31×31
Shubert (historical variant)	$[0, 10]$	$[0, 10]$	31×31
Styblinski–Tang	$[-5, 5]$	$[-5, 5]$	31×31
Sum of squares	$[-10, 10]$	$[-10, 10]$	31×31
Xin-She Yang 3 (historical variant)	$[-20, 20]$	$[-20, 20]$	31×31

S2 Grid-GDP objective functions

Each expression below defines a benchmark on the corresponding domain in Table S1, with all coefficients fixed. The definitions follow `gdp_models/grid.py`; differences from the historical `qhdopt_gdp.py` are noted where relevant (SECQUOIA, 2026b, n.d.b).

1. Ackley

$$f(x,y) = -20\exp\left(-0.2\sqrt{\frac{x^2+y^2}{2}}\right) - \exp\left(\frac{\cos(2\pi x)+\cos(2\pi y)}{2}\right) + 20 + e. \tag{S3}$$

2. Alpine 1

$$f(x,y) = |x\sin x + 0.1x| + |y\sin y + 0.1y|. \tag{S4}$$

3. Alpine 2

$$f(x,y) = -\sqrt{xy}\,\sin x \sin y. \tag{S5}$$

The negative sign and the domain $[0,1000]^2$ are retained.

4. Bohachevsky 2

$$f(x,y) = x^2 + 2y^2 - 0.3\cos(3\pi x)\cos(4\pi y) + 0.3. \tag{S6}$$

5. Three-hump camel

$$f(x,y) = 2x^2 - 1.05x^4 + \frac{x^6}{6} + xy + y^2. \tag{S7}$$

6. Csendes (squared variant)

$$f(x,y) = c(x) + c(y), \qquad c(t) = \begin{cases} t^6\left[2+\sin(1/t)\right]^2, & t \neq 0, \\ 0, & t = 0. \end{cases} \tag{S8}$$

The square applies to the entire oscillatory factor. The current GDP uses branch-fixed auxiliaries $r_t = 1/t$ for $t \neq 0$ and $r_t = 0$ for $t = 0$, evaluating $t^6[2+\sin(r_t)]^2$ without division by zero.

The archived `qhdopt_gdp.py` instead uses $c_\varepsilon(t) = t^6[2+\sin(1/(t+\varepsilon))]^2$ with $\varepsilon = 10^{-12}$ (SECQUOIA, n.d.b). This also gives zero at $t = 0$, but is not algebraically identical to Equation (S8) at nonzero coordinates. The current implementation uses the exact zero extension, with no epsilon shift; its normalized values agree with the archived grid to the stated 10^{-12} absolute verification tolerance.

7. Deflected corrugated spring

With $r = \sqrt{(x-5)^2 + (y-5)^2}$,

$$f(x,y) = 0.1r^2 - \cos(5r). \tag{S9}$$

8. Drop-Wave

$$f(x,y) = -\frac{1+\cos\left(12\sqrt{x^2+y^2}\right)}{0.5(x^2+y^2)+2}. \tag{S10}$$

9. Easom

$$f(x,y) = -\cos x \cos y\, \exp\left(-[(x-\pi)^2 + (y-\pi)^2]\right). \tag{S11}$$

10. Griewank

$$f(x,y) = \frac{x^2+y^2}{4000} - \cos x \cos\left(\frac{y}{\sqrt{2}}\right) + 1. \tag{S12}$$

11. Hölder table

$$f(x,y) = -\left|\sin x \cos y \, \exp\left(\left|1 - \frac{\sqrt{x^2+y^2}}{\pi}\right|\right)\right|. \tag{S13}$$

12. Hosaki

$$f(x,y) = \left(1 - 8x + 7x^2 - \frac{7}{3}x^3 + \frac{1}{4}x^4\right) y^2 \exp(-y). \tag{S14}$$

13. Lévy

Define $w_x = 1 + (x-1)/4$ and $w_y = 1 + (y-1)/4$. Then

$$\begin{aligned} f(x,y) = \; & \sin^2(\pi w_x) \\ & + (w_x - 1)^2\left[1 + 10\sin^2(\pi w_x + 1)\right] \\ & + (w_y - 1)^2\left[1 + \sin^2(2\pi w_y)\right]. \end{aligned} \tag{S15}$$

The middle sine argument is $\pi w_x + 1$.

14. Lévy 13

$$\begin{aligned} f(x,y) = \; & \sin^2(3\pi x) + (x-1)^2\left[1 + \sin^2(3\pi y)\right] \\ & + (y-1)^2\left[1 + \sin^2(2\pi y)\right]. \end{aligned} \tag{S16}$$

15. Michalewicz

$$f(x,y) = -\sin x \, \sin^{20}\left(\frac{x^2}{\pi}\right) - \sin y \, \sin^{20}\left(\frac{2y^2}{\pi}\right). \tag{S17}$$

The fixed exponent is 20 (equivalently $2m$ with $m = 10$).

16. Rastrigin

$$f(x,y) = x^2 - 10\cos(2\pi x) + y^2 - 10\cos(2\pi y) + 20. \tag{S18}$$

17. Rosenbrock

$$f(x,y) = (1-x)^2 + 100(y - x^2)^2. \tag{S19}$$

18. Shubert (historical variant)

$$\begin{aligned} f(x,y) = \; & \left[\cos(2x+1) + 2\cos(3x+2) + 3\cos(4x+3)\right] \\ & \times \left[\cos(2y+1) + \cos(y+2)\right]. \end{aligned} \tag{S20}$$

This is the implemented three-term by two-term product. In particular, the final term is $\cos(y+2)$ with coefficient one.

19. Styblinski–Tang

$$f(x,y) = \frac{1}{78}\,\frac{1}{2}\left(x^4 - 16x^2 + 5x + y^4 - 16y^2 + 5y\right). \tag{S21}$$

The historical factor $1/78$ is retained in the raw objective, even though positive scaling cancels in the normalization of Equation (S2).

20. Sum of squares

$$f(x, y) = x^2 + 2y^2. \tag{S22}$$

21. Xin-She Yang 3 (historical variant)

$$f(x, y) = \exp\left(-\left[\left(\frac{x}{15}\right)^6 + y^6\right]\right) + 2\exp\bigl(-(x^2 + y^2)\bigr)\cos^2 x \cos^2 y. \tag{S23}$$

The exponential–cosine term has a positive coefficient +2. Only x is divided by 15 in the first exponential; y^6 is unscaled. Floating-point exponential underflow can produce stored zero values; these do not change the analytical expression.

S3 Source verification

Exhaustive evaluation of the implemented Grid-GDP objectives at all $21 \times 961 = 20{,}181$ lattice points agrees with the archived `benchmark_grids/*.csv` normalized values within absolute tolerance 10^{-12}. The target sets at $F \leq 10^{-5}$ also agree.

S4 Executable protocol and implementation settings

Python and NumPy use the job seed, and `RBFOpt` also receives `rand_seed`. `BAM` exposes no separate native random-seed option: its process wrapper seeds Python/NumPy, while the Grid seed identifies the common initial design. `FMQA-SA` uses three NumPy `SeedSequence(seed)` streams for FM initialization, annealing, and uniform unseen-point fallback; each training or annealing invocation draws a seed from its stream. `PyTorch` uses deterministic CPU operations and one thread.

Repeated callbacks reuse the stored response and point cost. Unaccepted process evaluations return 10^6 to `D-SDA` and `RBFOpt`.

Grid `D-SDA` ranks the initial points and attempts successive descents until reaching the budget or known reference target. It enumerates the full L_∞ neighborhood as the lexicographic Cartesian product of $(-1, 0, 1)$, excluding the zero direction and out-of-bounds points; ties select the first point, and improvement must exceed 10^{-12}.

Primary `RBFOpt` 4.3.0 uses MSRSM, `rbf=auto`, the default genetic candidate-search option `global_search_method=genetic`, integer variables, and `do_init_strategy=False`. The recorded defaults are `ga_base_population_size=400` and `ga_num_generations=20`. `RBFOpt` 4.3.0 uses an effective population of $400 + \lfloor 20n/5 \rfloor$, where n is the dimension passed to the genetic-search routine, giving 408 and 412 for two and three search variables. Standalone `Bonmin` 1.8.9 (Bonami et al., 2008) solves surrogate MINLPs, and the configured `IPOPT` 3.14.19 executable supports surrogate NLPs (Wächter and Biegler, 2006). `BAM` 5.0.0 receives `x0=None`, all-integer variables, the three evaluated starts, and `max_evals`=B-3, with other options at package defaults; its local models use `ALAMO`/`BARON`.

`FMQA-SA` stops at B or its 1,800 s method limit, checked between operations and through training/sampling callbacks, without reference-target stopping. All process workers and the Grid `FMQA-SA` worker have a 2,100 s watchdog. The Grid `RBFOpt`/`BAM` trace watchdogs are disabled (`max_run_seconds=0`); archived Grid `D-SDA` uses budget, target, or local-search termination without an additional method-clock limit.

S4.1 Versions and independently timed process evaluations

The recorded environments include Python 3.11.16, `RBFOpt` 4.3.0 (Costa and Nannicini, 2018; Nannicini, 2021), `BAM` 5.0.0 (Ma et al., 2023), `Pyomo` 6.10.0 (Bynum et al., 2021), `BARON` 25.11.17, and `HiGHS` 1.12.0 (Huangfu and Hall, 2018). The annealer is the classical CPU implementation `dwave.samplers.SimulatedAnnealingSampler` (D-Wave Systems, n.d.). `BAM`'s internal `ALAMO` and `BARON` versions are unspecified; `BARON` 25.11.17 is used for the algebraic comparisons.

The fixed-configuration process evaluations use IPOPT 3.14.19 (Wächter and Biegler, 2006) with MUMPS 5.8.1 (Amestoy et al., 2001), solver tolerance 10^{-8} and a 60 s solver limit. The acceptance rule in gdp_models/evaluation.py requires all of the following: a finite objective; termination in {optimal, locallyOptimal, globallyOptimal}; solver status other than error or aborted; interface status SOLVESTAT=1 and MODELSTAT$\in \{1, 2\}$; and a finite maximum primal violation no greater than 10^{-5}. The 10^{-8} solver tolerance is distinct from this acceptance threshold. Time-limit, iteration-limit, evaluation-limit, and error outcomes are not accepted, even if a finite feasible candidate is available.

The primal check takes the largest positive absolute constraint-bound or variable-bound violation, with zero for satisfied bounds and no row-wise normalization. Constraint rows are collected after fixing the disjunctions and before preprocessing, so rows subsequently deactivated by presolve are also checked. Bounds of initialized active variables are checked on the prepared NLP. An undefined or non-finite constraint body, a non-finite initialized variable, or a non-finite objective precludes acceptance. Uninitialized unused variables are skipped; the objective and all retained constraint expressions are still evaluated. Preprocessing rejection or worker failure supplies no accepted objective.

CSTR5–CSTR30 start from constructor defaults and then apply cstr_box_ipopt.py:initialize_feasible after preprocessing. This sets mass-balance-consistent values with zero recycle and checks the initial primal violation at 10^{-8} without changing constraints or bounds. The CSTR30 table retains one interface failure, which none of the 50 searches visited. Column and Small Batch load gdp/dsda/column_17_initialization.json and gdp/dsda/small_batch_initialization.json, respectively, from the recorded dsda-gdp source revision, using its gdp/dsda/model_serializer.py. Dynamic clones the unsolved discretized template; it has no saved initialization vector. These are cold starts, with no transfer of an NLP solution between configurations. Objectives, acceptance status, and NLP point costs are linked to the same recorded attempt by model, CSTR size, and coordinate, without successful-retry selection or cross-size time substitution. The CSTR30 point costs include initialization and artifact recording. Each stored cost is reused on revisits, so reconstructed costs do not capture repeat-measurement timing variability; Dynamic template preparation is added once per run.

S4.2 Observed proposal and initialization behavior

Table S2 pools FMQA-SA proposals across tested budgets and ten seeds per condition, including all six CSTR sizes. Counts exclude the shared initialization and include terminal-callback proposals; rates are pooled fallback/proposal ratios. Uniform fallback supplies most process proposals, so the results characterize the combined learned-proposal and fallback procedure. The added uniform control compares the complete FMQA-SA procedure with sampling under the shared protocol; it does not isolate the individual causal contributions of fitting, annealing, and fallback.

Table S2: Recorded FMQA-SA uniform fallback by benchmark family. Counts include proposals for terminal callbacks.

Family	Runs	Proposals	Fallbacks	Rate
Grid	1,050	51,450	9,522	18.51%
CSTR	60	1,350	369	27.33%
Column	10	130	82	63.08%
Dynamic	60	1,460	1,063	72.81%
Small Batch	10	70	58	82.86%
Process total	140	3,010	1,572	52.23%

S4.3 Terminal-callback solutions and final summaries

Process success is defined by the wrapper's best accepted configuration, including callback B, rather than by the native optimizer's returned incumbent.

S5 Process model scale, units, and discrete decisions

Counts below describe the unfixed original GDP before reformulation or presolve, including all active disjunct branches, not the smaller NLP for a particular selected configuration. Fixed continuous variables remain in the continuous-variable count; the numerical suffix in `CSTR5`–`CSTR30` specifies the number of potential reactor positions in the superstructure. `Dynamic` is discretized with 30 finite elements per stage and three Radau points. Binary counts refer to the indicator variables already constructed by the model, before Boolean-to-binary conversion. All nine models have zero additional nonbinary integer variables; the process GDP formulations appear in Appendices A.1–A.3 of Ovalle et al. (2025), with external-variable mappings in Appendices B.1, B.2, and B.4, and the `Dynamic` formulation appears in Section IV-C of Peng et al. (2024).

Table S3: Original process-model sizes. n_c: continuous variables; n_b: binary variables; m_a: algebraic constraints; m_l: logical constraints; n_d: disjunctions.

Model	n_c	n_b	m_a	m_l	n_d
`CSTR5`	56	20	100	12	10
`CSTR10`	106	40	195	22	20
`CSTR15`	156	60	290	32	30
`CSTR20`	206	80	385	42	40
`CSTR25`	256	100	480	52	50
`CSTR30`	306	120	575	62	60
`Column`	446	30	637	16	15
`Small Batch`	19	18	34	3	9
`Dynamic`	1647	27	3276	31	9

For `CSTR`, the cutoff/feed and recycle choices determine reactor/bypass and recycle disjuncts; the objective is total reactor volume in litres (Liñán et al., 2020). `Column` selects reflux/boilup positions and the implied tray configuration (Ghouse et al., 2018). Its implemented objective is $10^3(Q_c + Q_b) + 10^3 n_{\text{active}}$, a weighted cost proxy labelled in dollars by the source model; it is not a newly estimated annualized economic cost. `Small Batch` selects the parallel-unit counts for three stages, fixing the discrete unit-number alternatives; its source objective is investment cost in dollars (Kocis and Grossmann, 1988). No currency-year conversion is applied. `Dynamic`'s two switching-stage choices fix each stage's mode and leave the continuous state trajectories to the discretized NLP (Peng et al., 2024). Its objective is $-\int_0^9 x(t)^2\,dt$, so minimization favors a larger integrated squared state. State and time are abstract benchmark quantities without a physical unit calibration in the implementation. The example adds mode sequencing and dynamic differential constraints to the static equipment/configuration cases; it is not a plant-specific economic metric.

`Column` uses the predeclared historical lookup-study box $\{7, \ldots, 15\} \times \{1, \ldots, 9\}$ (81 points). This is a restricted comparison domain within the full 225-point GDP box, not a proof that all outside configurations are infeasible. The box was chosen as an experimental scope restriction without a specific engineering or optimization-based selection criterion. Consequently, the DFO findings are conditional on this restricted domain. All four DFO methods use the same box; the algebraic references use the original GDP domain.

S6 Grid inventory and target-set sizes

All DFO methods receive the integer pair $(i, j) \in \{0, \ldots, 30\}^2$. The one-based external-variable convention in the main paper corresponds to $(\xi_1, \xi_2) = (i + 1, j + 1)$; this is a relabelling, not an offset applied by the Grid optimizers. Physical coordinates follow Equation (S1). `FMQA-SA` uses five unsigned bits per Grid coordinate; decoded coordinate values equal to 31 are rejected as out of bounds. The normalized tolerance 10^{-5} equals $10^{-5}(U - L)$ in raw objective units. Target-set sizes therefore vary across functions even though the normalized tolerance is shared.

The inventory in Table S4 is calculated from all 961 implemented raw samples. Constants are rounded for display; full precision, physical coordinates, and complete minimum/target lists are in `grid_inventory.json`. $n_{\text{min}}^{\text{fp}}$ counts exact ties at the smallest floating-point raw sample, and n_{target} counts normalized samples at most 10^{-5}. Floating arithmetic may split analytic ties: Rosenbrock has the two exact minimizers stated

in the main paper, although only one sample equals the floating minimum bit for bit. For Xin-She Yang 3 (historical variant), underflow gives 12 exact stored minima, and 870 grid points meet the target. Neither exact stored zeros nor target membership imply that the corresponding continuous analytical function is zero there.

Table S4: Enumerated grid normalization constants and target-set sizes.

Function	L	U	$n_{\min}^{\mathrm{fp}}$	n_{target}
Ackley	4.440892e-16	14.24694	1	1
Alpine 1	0	17.42973	1	1
Alpine 2	-896.0502	789.5396	1	1
Bohachevsky 2	0	30000	1	1
Three-hump camel	0	2047.917	1	1
Csendes (squared variant)	0	16.14791	1	36
Deflected corrugated spring	-1	5.698269	1	1
Drop-Wave	-1	-1.314513e-07	1	1
Easom	-0.8953676	0.008461561	1	1
Griewank	0	180.0121	1	1
Hölder table	-18.13802	-0	2	4
Hosaki	-2.345812	0.5413411	1	1
Lévy	0.1140618	95.38281	1	2
Lévy 13	0.3055556	307.8611	1	4
Michalewicz	-1.80107	-0	1	1
Rastrigin	0	77.84942	1	1
Rosenbrock	0.006419753	2509	1	2
Shubert (historical variant)	-11.03782	10.72027	1	1
Styblinski–Tang	-1	3.205128	1	1
Sum of squares	0	300	1	1
Xin-She Yang 3 (historical variant)	0	3	12	870

S7 Success-rate uncertainty and completion costs

For each problem, method, and budget, let h be the number of target hits among $n = 10$ seeds, retaining execution failures as non-successes.

We report two-sided 95% Clopper–Pearson intervals for the binomial success probability (Clopper and Pearson, 1934):

$$p_L = \begin{cases} 0, & h = 0, \\ B^{-1}_{0.025}(h, n-h+1), & h > 0, \end{cases} \qquad p_U = \begin{cases} 1, & h = n, \\ B^{-1}_{0.975}(h+1, n-h), & h < n, \end{cases}$$

where B^{-1} denotes the beta quantile. Coverage is conditional on treating the seed-level outcomes as independent Bernoulli trials under the fixed protocol. The intervals are pointwise, not simultaneous across comparisons.

Ten observed successes give $[0.6915, 1]$; zero give $[0, 0.3085]$. Thus neither empirical endpoint establishes a population probability of one or zero. Table S5 reports completion costs for all 14 process conditions.

Table S6 reports function-level results at the largest Grid budget, $B = 100$. Complete seed-level outcomes for all Grid budgets and Process conditions are available in the publication analysis linked in Section S8. Cells report $h\,[p_L, p_U]$, with limits rounded to two decimals; displayed endpoints may be rounded interior bounds. Seed indices remain paired across methods within each problem/budget, but intervals are calculated separately for each group, not by pooling the heterogeneous 21-function suite.

Table S5: Median reconstructed full-run completion costs for all ten paired seeds per condition, including misses and timed execution failures, in seconds.

Case	B	D-SDA	RBFOpt	BAM	FMQA-SA	Uniform
Column	16	37.91	33.54	37.97	29.72	26.29
CSTR5	8	10.66	10.72	10.67	13.92	9.44
CSTR10	15	20.98	21.79	18.54	27.07	17.75
CSTR15	22	29.92	37.59	27.67	42.82	27.84
CSTR20	29	40.67	45.03	39.37	59.10	40.12
CSTR25	36	51.67	62.29	50.47	74.70	51.17
CSTR30	43	53.08	101.96	115.41	101.19	65.20
Small Batch	10	3.83	5.12	4.23	9.09	3.49
Dynamic	12	12.22	12.29	9.95	14.47	7.80
Dynamic	15	15.84	16.10	11.53	18.39	8.71
Dynamic	20	21.29	20.54	14.07	25.23	12.27
Dynamic	30	26.61	34.19	21.91	36.24	19.38
Dynamic	37	26.60	42.42	24.99	46.05	24.38
Dynamic	50	26.60	65.14	33.98	61.31	33.12

The table reports completion costs, not first-hit times. Reconstructed search-completion costs exclude the final wrapper result-extraction and summary-writing pass. Pointwise success-rate intervals are defined in Supplementary Section S7. Every group has complete measured point-cost coverage, with no zero filling or cross-size substitution.

Table S6: Grid target hits and pointwise 95% success-probability intervals at $B = 100$. Each cell is $h\,[p_L, p_U]$ from ten seeds.

Function	D-SDA	RBFOpt	BAM	FMQA-SA	Uniform	BAM-prefix control
Ackley	0 [0.00, 0.31]	10 [0.69, 1.00]	10 [0.69, 1.00]	8 [0.44, 0.97]	1 [0.00, 0.45]	10 [0.69, 1.00]
Alpine1	1 [0.00, 0.45]	10 [0.69, 1.00]	10 [0.69, 1.00]	5 [0.19, 0.81]	1 [0.00, 0.45]	10 [0.69, 1.00]
Alpine2	1 [0.00, 0.45]	3 [0.07, 0.65]	1 [0.00, 0.45]	2 [0.03, 0.56]	1 [0.00, 0.45]	2 [0.03, 0.56]
Bohachevsky2	10 [0.69, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]	9 [0.55, 1.00]	1 [0.00, 0.45]	10 [0.69, 1.00]
Camel3	9 [0.55, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]	3 [0.07, 0.65]	2 [0.03, 0.56]	10 [0.69, 1.00]
Csendes	10 [0.69, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]
Deflected corrugated spring	0 [0.00, 0.31]	1 [0.00, 0.45]	10 [0.69, 1.00]	3 [0.07, 0.65]	3 [0.07, 0.65]	10 [0.69, 1.00]
Dropwave	0 [0.00, 0.31]	2 [0.03, 0.56]	10 [0.69, 1.00]	4 [0.12, 0.74]	0 [0.00, 0.31]	10 [0.69, 1.00]
Easom	3 [0.07, 0.65]	7 [0.35, 0.93]	0 [0.00, 0.31]	3 [0.07, 0.65]	0 [0.00, 0.31]	1 [0.00, 0.45]
Griewank	10 [0.69, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]	2 [0.03, 0.56]	10 [0.69, 1.00]
Holder Table	1 [0.00, 0.45]	10 [0.69, 1.00]	6 [0.26, 0.88]	8 [0.44, 0.97]	4 [0.12, 0.74]	1 [0.00, 0.45]
Hosaki	10 [0.69, 1.00]	10 [0.69, 1.00]	3 [0.07, 0.65]	6 [0.26, 0.88]	0 [0.00, 0.31]	3 [0.07, 0.65]
Levy	2 [0.03, 0.56]	10 [0.69, 1.00]	5 [0.19, 0.81]	4 [0.12, 0.74]	0 [0.00, 0.31]	2 [0.03, 0.56]
Levy13	8 [0.44, 0.97]	10 [0.69, 1.00]	9 [0.55, 1.00]	9 [0.55, 1.00]	6 [0.26, 0.88]	2 [0.03, 0.56]
Michalewicz	7 [0.35, 0.93]	10 [0.69, 1.00]	2 [0.03, 0.56]	7 [0.35, 0.93]	0 [0.00, 0.31]	2 [0.03, 0.56]
Rastrigin	0 [0.00, 0.31]	4 [0.12, 0.74]	10 [0.69, 1.00]	5 [0.19, 0.81]	1 [0.00, 0.45]	10 [0.69, 1.00]
Rosenbrock	3 [0.07, 0.65]	10 [0.69, 1.00]	0 [0.00, 0.31]	6 [0.26, 0.88]	2 [0.03, 0.56]	1 [0.00, 0.45]
Shubert	1 [0.00, 0.45]	4 [0.12, 0.74]	1 [0.00, 0.45]	1 [0.00, 0.45]	1 [0.00, 0.45]	0 [0.00, 0.31]
Styblinski Tang	8 [0.44, 0.97]	10 [0.69, 1.00]	10 [0.69, 1.00]	7 [0.35, 0.93]	0 [0.00, 0.31]	0 [0.00, 0.31]
Sum Of Squares	10 [0.69, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]	9 [0.55, 1.00]	0 [0.00, 0.31]	10 [0.69, 1.00]
Xin She Yang3	10 [0.69, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]	10 [0.69, 1.00]

S8 Data sources and reproducibility

The baseline publication analysis provides the original source records and reproduction instructions (SECQUOIA, 2026e). The CSTR5–25 results use the finite-penalty records for `D-SDA` and `RBFOpt`. The `CSTR30` results use the updated evaluation table and all 50 searches in the initialization follow-up. The resulting 700-run process comparison contains 2,040 distinct visited configurations and 16,206 callbacks. The updated figures and tables are generated by the publication analysis.

S9 Feasibility and bound consistency of the algebraic results

The bound-consistency check covers all 45 selected algebraic runs in manuscript Table 6. Table S7 lists the 11 flagged comparisons, and Table S8 gives the nine saved feasible comparison solutions.

For each minimization model, let f_w be the recorded objective of its best independently checked saved feasible solution, and let L denote a reported lower bound. We apply the same rule to all five methods:

$$\tau_w = 10^{-5} \max\{1, |f_w|\}, \qquad \Delta = L - f_w, \qquad \rho = \Delta/\tau_w, \qquad \dagger \iff \rho > 1. \tag{S24}$$

Missing lower bounds remain missing, not zero.

Saved vectors are loaded into the original GDP to reevaluate objectives and check active constraints, variable bounds, integrality, disjunctions, and logical validity without optimization. Violations are unscaled original-equation residuals, not reformulation residuals. All selected-run and comparison-solution vectors have defined expressions and satisfy the 10^{-5} feasibility threshold. This numerical check establishes neither exact feasibility nor global optimality.

The uniform rule identifies 11 bound conflicts: the six `CSTR` GLOA runs, `Column` Big-M, and `CSTR5`, `CSTR10`, `CSTR15`, and `CSTR30` Hull. These conflicting bounds and `optimal` reports provide no independent global certificate or intrinsic method ranking; their cause in GDPopt or its subsolvers remains unresolved.

Table S7: The 11 flagged bound comparisons among the 45 selected runs in manuscript Table 6. Lower bounds retain their stored digits; $v_{\max}$ and ρ are rounded for display. A dagger is assigned using the unrounded $\rho > 1$ test, and W denotes the witness ID in Table S8. These flags are numerical-consistency categories, not equally strong demonstrations of invalid mathematical bounds. The smaller Hull discrepancies for `CSTR5`, `CSTR10`, and `CSTR15` are particularly sensitive to numerical feasibility and tolerance conventions.

ID	Model	Method	Reported L	$v_{\max}$	W	ρ
A02	`CSTR5`	GLOA	3.1301981330832502	1.425e-11	W01	2227.78†
A05	`CSTR5`	Hull	3.06201457880245	6.836e-10	W01	1.00006†
A07	`CSTR10`	GLOA	3.44796403174898	2.169e-11	W02	19328†
A10	`CSTR10`	Hull	2.889535836078951	5.265e-07	W02	1.81291†
A12	`CSTR15`	GLOA	3.522552007803194	1.306e-07	W03	24500.4†
A15	`CSTR15`	Hull	2.829394077848967	1.101e-09	W03	1.53603†
A17	`CSTR20`	GLOA	3.6161229932346584	8.316e-12	W04	29174.6†
A22	`CSTR25`	GLOA	3.647818131769651	8.947e-07	W05	31146.5†
A27	`CSTR30`	GLOA	3.154308073141742	4.528e-08	W06	13891.6†
A30	`CSTR30`	Hull	2.809386231078974	8.796e-07	W06	1437.64†
A34	`Column`	Big-M	19853.722966464	1.009e-07	W07	2623.77†

Table S8: Saved feasible comparison solutions. The CSV links to each full vector and includes original archive/member paths and hashes, reevaluated objective, and unrounded violation.

ID	Model	Method	Recorded f_w	$v_{\max}$	Source
W01	`CSTR5`	RIC	3.06198395705692	6.645e-07	R2
W02	`CSTR10`	Big-M	2.8894834524326347	3.941e-07	R1
W03	`CSTR15`	LOA	2.82935061823614	5.634e-07	R3
W04	`CSTR20`	Big-M	2.7994063086813563	8.937e-10	R1
W05	`CSTR25`	Hull	2.781482618820502	6.445e-10	R1
W06	`CSTR30`	Big-M	2.7695697953461833	9.948e-09	R1
W07	`Column`	RIC	19346.124801099686	8.527e-14	R1
W08	`Small Batch`	Hull	167427.6569847343	5.902e-07	R1
W09	`Dynamic`	RIC	-212.00878715666482	8.313e-13	R1

R1: `results/gdp_comparison_20260925/raw`.
R2: `results/gdp_comparison_20261001/raw`.
R3: `results/gdp_comparison_loa_20261001/raw`.

Additional tightened-feasibility check for `CSTR15`. A separate Dantzig `IPOPT` 3.14.19 solve with `MUMPS` 5.8.1 retained W03's discrete configuration and continuous starting values and targeted an independently checked original-GDP residual of 10^{-8} (SECQUOIA, 2026c). Only exactly satisfied constant rows were omitted from the computational clone, preserving the fixed-configuration feasible set and all original constraints in the independent check. The solve returned objective 2.8293940632857595. The original-GDP recheck gave maximum unscaled violation $1.1102230246251565 \times 10^{-16}$, passing the 10^{-8} target.

The recorded A12 GLOA lower bound 3.522552007803194 exceeds the tightened objective by 0.6931579445174347, approximately 24,498 times the unchanged comparison tolerance $10^{-5}\max\{1, |f|\}$, equal to $2.8293940632857597 \times 10^{-5}$. For Hull, the recorded lower bound 2.829394077848967 exceeds this objective by only 1.45632×10^{-8}, giving $\rho \approx 5.14711 \times 10^{-4} < 1$ and no flag for this comparison, versus $\rho \approx 1.536$ with historical witness W03. This result establishes neither exact feasibility, global optimality, the bound discrepancy's cause nor conclusions for other cases.

S10 Additional candidate-search and sampling comparisons

S10.1 Paired differences and sampling controls

Paired success differences retain the common case, budget and seed. For Grid, the descriptive percentile bootstrap resamples the 21 intact function vectors, each containing ten paired seed differences, rather than treating 210 observations as exchangeable Bernoulli draws. For Process, paired seeds are resampled within each of the 14 conditions; the six related `Dynamic` budgets are not pooled for inference. The seed is 20261020 with 2,000 bootstrap replications and deterministic case/seed ordering. Intervals are exploratory and unadjusted for multiple comparisons; ten-seed boundary cases may yield degenerate intervals, and zero inclusion does not establish equivalence. Full condition-level comparisons are in `manuscript/tables/paired_comparisons.csv`, with the independently preserved FMQA–uniform analysis under `results/review_comparison_20261020/fmqa_uniform/`.

Table S9: Selected paired success differences, in percentage points. The first method minus the second defines the sign; Process intervals concern the stated condition only.

Grid

B	Contrast	Difference	95% interval
10	`RBFOpt` – Uniform	13.33	[5.71, 24.29]
100	`RBFOpt` – Uniform	60.00	[42.86, 75.71]
10	`BAM` – `BAM`-prefix control	0.00	[0.00, 0.00]
100	`BAM` – `BAM`-prefix control	10.95	[0.48, 23.81]
10	`FMQA-SA` – Uniform	0.95	[-0.95, 2.86]
100	`FMQA-SA` – Uniform	40.00	[28.57, 52.38]
10	`D-SDA` – `RBFOpt`	-13.81	[-24.29, -6.19]
100	`D-SDA` – `RBFOpt`	-31.90	[-46.19, -18.10]

Process: ten paired seeds per condition

Case	B	Contrast	Outcomes	Difference	95% interval
`Dynamic`	12	`RBFOpt` – `D-SDA`	4/6/0/0	60	[30, 90]
`Small Batch`	10	`FMQA-SA` – Uniform	2/4/2/2	20	[-30, 60]

Outcomes list both successful / first only / second only / neither successful. The descriptive intervals resample functions for Grid and paired seeds within each Process condition; they do not establish general superiority or equivalence.

S10.2 Grid suite exclusions

Excluding Xin-She Yang 3 and Bohachevsky 2 preserves the main descriptive pattern. For the sampled Bohachevsky 2 lattice, $\cos(3\pi a_i) = 1$ at every coordinate, leaving an objective nearly redundant with sampled Sum of Squares after normalization. Without both functions, B=100 rates are 79.47% for `RBFOpt`, 66.84% for `BAM`, 57.89% for `FMQA-SA`, 54.74% for the `BAM`-prefix control, 44.21% for `D-SDA`, and 17.89% for uniform. At B=10, `BAM` and its prefix control each retain 47.89%, compared with `RBFOpt` 15.79%.

S10.3 Limited reconstructed FMQA diagnostics

Seven FM models were reconstructed locally from archived observations in three selected runs because the original fitted weights and QUBOs were not saved (SECQUOIA, 2026d). All 3,328 enumerated binary states passed the recorded energy-conversion checks. Five selected SA proposals minimized their reconstructed models over eligible candidates; two `Dynamic` fallback iterations had no eligible stored SA sample. These selected checks do not verify the historical weights or isolate the contributions of learning, annealing, and fallback. Synthetic zero-coefficient tests produced nonfinite scaling and a wrapper error before annealing or fallback; whether this occurred in the historical fits is unknown.